Rockchip RK3588 USB Developer Guide

Document Identifier: RK-SM-YF-450

Release Version: V1.3.0

Release Date: 2024-10-09

Security Level: □Top-Secret □Secret □Internal ■Public

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document provides a development guide for the RK3588 USB module, aimed at helping developers understand the hardware circuit design and software DTS configuration of the RK3588 USB controller and PHY, so that developers can flexibly design and quickly develop according to the USB application needs of the product.

Chipset	Kernel Version
RK3588, RK3588S	Linux-5.10 and above versions

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Wu Liangfeng	2021-01-18	Initial version
V1.1.0	Wu Liangfeng Wang Mingcheng	2022-09-22	Added Micro interface configuration instructions Added DT important attribute explanations
V1.2.0	Wu Liangfeng	2024-04-24	Added DTS attributes and explanations for USB2 only scheme
V1.3.0	Wu Liangfeng	2024-10-09	Modified the list of Type-C controller chip support, added support for AW35615

Contents

1. RK3588 USB Controller and PHY Introduction

RK3588 supports 5 independent USB controllers, including: 2 USB 2.0 HOST controllers, 2 USB 3.1 OTG controllers, and 1 USB 3.1 HOST controller. RK3588S has one less USB 3.1 OTG controller compared to RK3588. The specific types of USB controllers are shown in Table 1 below. For more detailed characteristics of the USB controllers, please refer to the RK3588 datasheet.

Table 1 RK3588 USB Controller List

Chip/Controller	USB 2.0 HOST (EHCI&OHCI)	USB 3.1 OTG (DWC3&xHCI)	USB 3.1 Host(xHCI)
RK3588	2	2	1
RK3588S	2	1	1

RK3588 supports 7 independent USB PHYs, including: 4 USB 2.0 PHYs, 2 USB 3.1/DP Combo PHYs, and 1 USB 3.1/SATA/PCIe Combo PHY. RK3588S has one less USB 2.0 PHY and one less USB 3.1/DP Combo PHY compared to RK3588.

Table 2 RK3588 USB PHY Support List

Chip/PHY	USB 2.0 PHY	USB3.1/DP ComboPHY	USB 3.1/SATA/PCIe ComboPHY
RK3588	4 [1 × port]	2	1
RK3588S	3 [1 × port]	1	1

Note:

1. In the table, the number N indicates support for N independent USB controllers and USB PHYs;
2. In the table, [1 × ports] indicates that one PHY supports only 1 USB port;
3. In the table, "EHCI&OHCI" indicates that the USB controller integrates an EHCI controller and an OHCI controller. "DWC3&xHCI" indicates that the USB controller integrates a DWC3 controller and an xHCI controller;
4. The USB 3.1 Gen1 physical layer transfer rate is 5Gbps, and the USB 2.0 physical layer transfer rate is 480Mbps;
5. The USB 3.1/DP Combo PHY supports 4 x lanes, which can simultaneously support USB 3.1 + DP 2 x lanes;
6. The USB 3.1/SATA/PCIe Combo PHY can only support one working mode at a time, meaning that the USB3.1 interface is mutually exclusive with the SATA/PCIe interface;

Table 3 RK3588 USB Controller and PHY Connection Relationship

USB Interface Name (Schematic)	USB Controller	USB PHY
TYPEC0	OTG0 (DWC3&xHCI)	USB3.1/DP ComboPHY0 + USB2.0 PHY0
TYPEC1	OTG1 (DWC3&xHCI)	USB3.1/DP ComboPHY1 + USB2.0 PHY1
USB20_HOST0	USB2.0 HOST0 (EHCI&OHCI)	USB2.0 PHY2
USB20_HOST1	USB2.0 HOST1 (EHCI&OHCI)	USB2.0 PHY3
USB30_2	USB3.1 HOST2 (xHCI)	USB3.1/SATA/PCIe ComboPHY2

RK3588 USB Controller and chip-side USB data transfer pin correspondence is shown in Table 4 below.

Table 4 RK3588 USB Controller and USB Pin Correspondence

USB Controller/Pin	RK3588 USB data pin
USB 2.0 HOST0	USB20_HOST0_DP/USB20_HOST0_DM
USB 2.0 HOST1	USB20_HOST1_DP/USB20_HOST1_DM
USB 3.1 OTG0	TYPEC0_USB20_OTG_DP/TYPEC0_USB20_OTG_DM, TYPEC0_SSRX1P/TYPEC0_SSRX1N, TYPEC0_SSTX1P/TYPEC0_SSTX1N, TYPEC0_SSRX2P/TYPEC0_SSRX2N, TYPEC0_SSTX2P/TYPEC0_SSTX2N
USB 3.1 OTG1	TYPEC1_USB20_OTG_DP/TYPEC1_USB20_OTG_DM, TYPEC1_SSRX1P/TYPEC1_SSRX1N, TYPEC1_SSTX1P/TYPEC1_SSTX1N TYPEC1_SSRX2P/TYPEC1_SSRX2N, TYPEC1_SSTX2P/TYPEC1_SSTX2N
USB 3.1 HOST2	USB30_2_SSTXP/USB30_2_SSTXN USB30_2_SSRXP/USB30_2_SSRXN

The internal connection relationship of RK3588 USB controllers and PHYs, as well as the corresponding common USB physical interfaces, is shown in Figure 1 below.

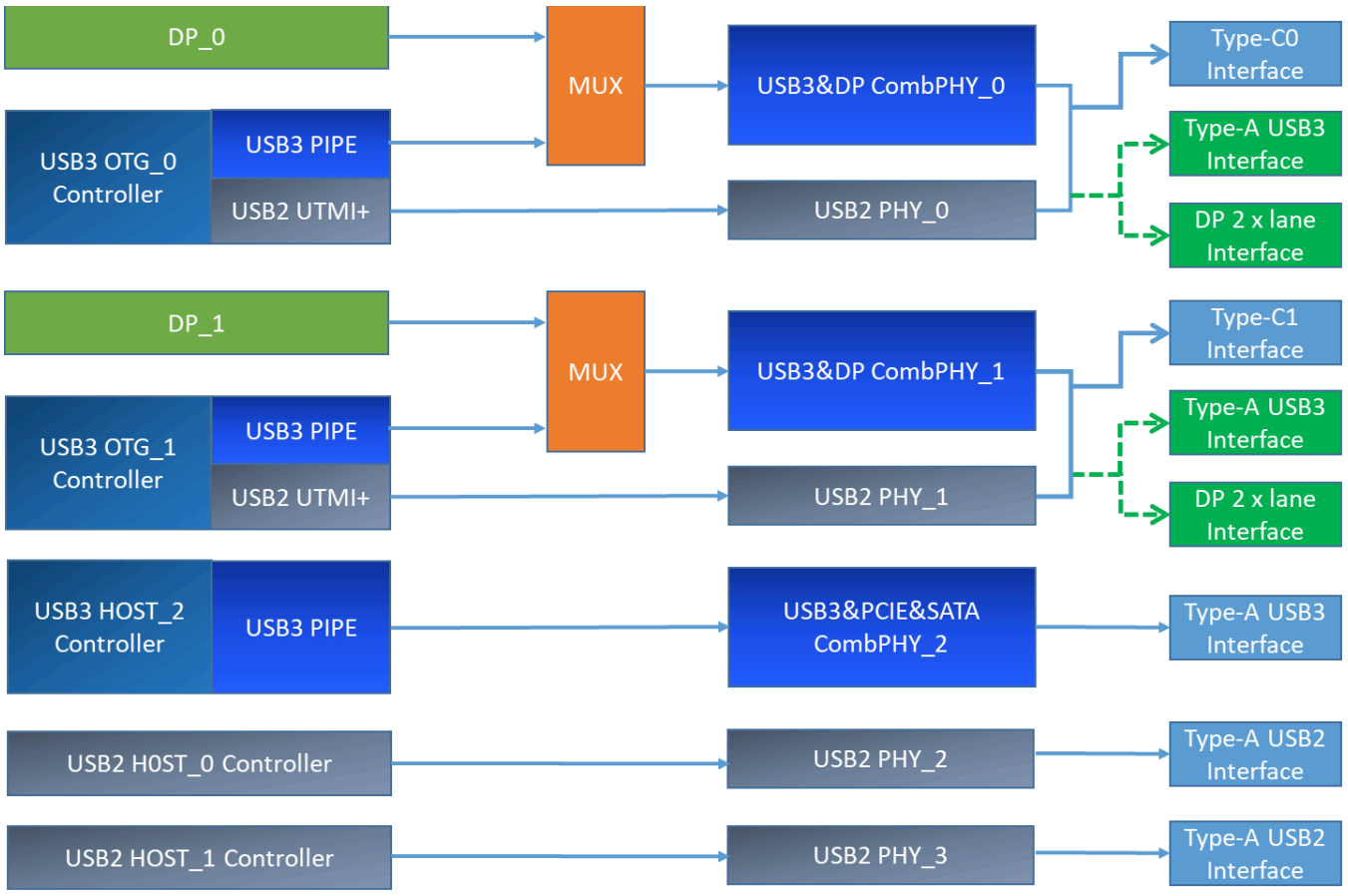

Figure 1 RK3588 USB Controller and PHY Connection Diagram

From Figure 1, it can be seen:

1. RK3588 can support up to 2 fully functional Type-C interfaces, 2 Type-A USB 2.0 interfaces, and 1 Type-A USB 3.1 only interface (not backward compatible with USB 2.0);
2. The USB 3.1 OTG controller and DP controller share the USB3.1/DP Combo PHY, which can form a fully functional Type-C interface or be used separately (e.g., common Type-A USB 3.1 interface + DP interface [2 x lanes]);
3. The USB 3.1 HOST_2 controller only supports USB 3.1 functionality and cannot support USB 2.0 (because it does not have a USB 2.0 PHY). USB 3.1 HOST2 can be combined with USB 2.0 HOST_0 or USB 2.0 HOST_1 to form a complete Type-A USB 3.1 interface;
4. The USB 3.1/DP Combo PHY can only support USB 3.1 Gen1 and is not backward compatible with USB 2.0. Therefore, in the chip's internal design, it is actually combined with the USB 2.0 PHY to support the complete USB 3.1 protocol functionality. Among them, USB 3.1/DP Combo PHY0 is fixed in combination with USB2 PHY0, and USB 3.1/DP Combo PHY1 is fixed in combination with USB2 PHY1;
5. The USB 3.1/SATA/PCIe Combo PHY can only support USB 3.1 Gen1 and is not backward compatible with USB 2.0. Moreover, in the chip's internal design, it is not combined with the USB2 PHY. Therefore, USB3 HOST_2 needs to be combined with the USB2 HOST_0/1 interface (choose one) to support the complete USB 3.1 protocol functionality;
6. The difference between the USB modules of RK3588 and RK3588S is that RK3588S has one less Type-C1 (i.e., 1 USB 3.1 OTG controller + 1 DP controller + 1 USB3.1/DP Combo PHY + 1 USB 2.0 PHY) compared to RK3588;

It should be noted that the types of interfaces supported by RK3588 USB are not limited to the Type-C/A USB interface types described in Figure 1, but can support all common USB interfaces, including Type-C USB 2.0/3.1, Type-A USB 2.0/3.1, Micro USB 2.0/3.1, etc. For specific information, please refer to RK3588 USB Supported Interface Types. To adapt to different USB circuit designs and interface types, the Linux-5.10 kernel USB driver has been software compatible. Developers only need to correctly configure the Linux USB DTS according to the product's USB hardware circuit to enable the corresponding USB interface functionality. For detailed USB DTS configuration methods, please refer to RK3588 USB DTS Configuration.

2. RK3588 USB Supported Interface Types

The RK3588 USB can support the common USB interface types as shown in Figure 2 below. When designing a product, the USB hardware circuit can be flexibly designed according to the actual application scenario requirements, and as long as the Linux USB DTS is adapted.

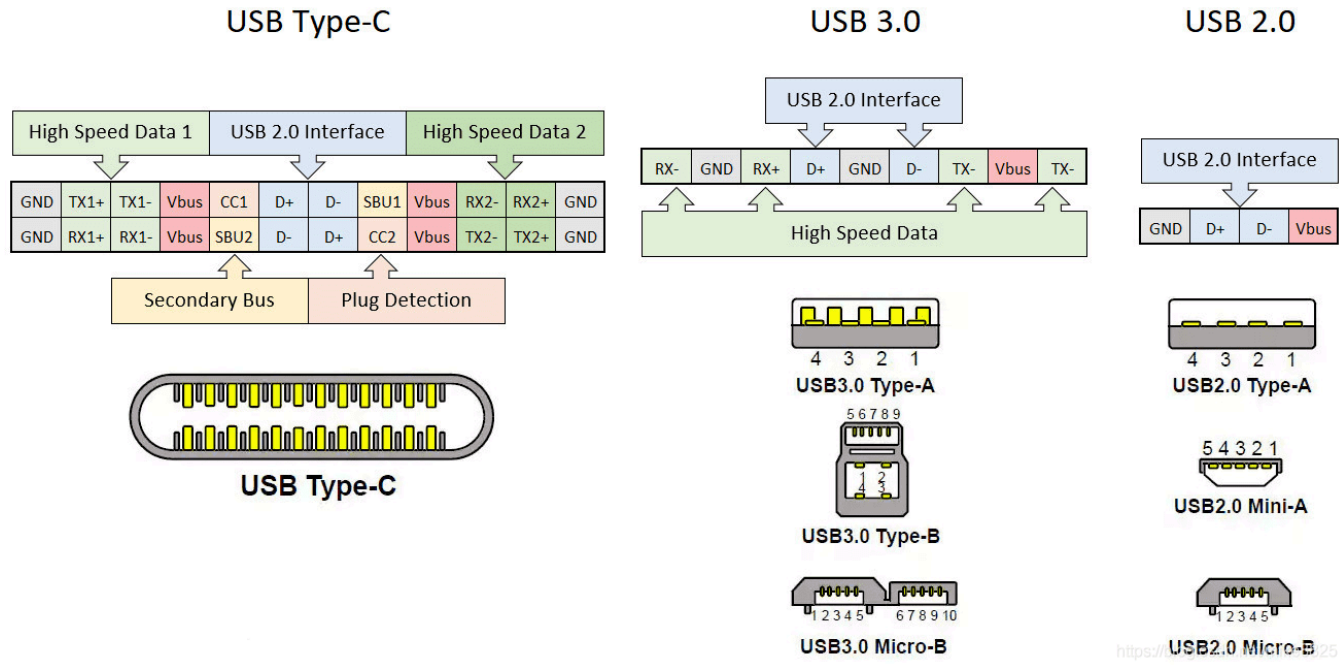

Figure 2 USB Interface Types

2.1 Type-C Interface Types

2.1.1 Type-C USB 3.1/DP Full-Function Interface

RK3588 Type-C0/1 can support the full functionality of Type-C interfaces[1], as shown in Figure 3 below, for specific hardware circuit design, please refer to Type-C USB 3.1/DP Full-Function Hardware Circuit. The main supported features are as follows:

- Support for Type-C PD (requires an external Type-C Controller Chip)
- Support for USB 3.1 Gen1 5Gbps data transfer
- Support for DP Alternate Mode

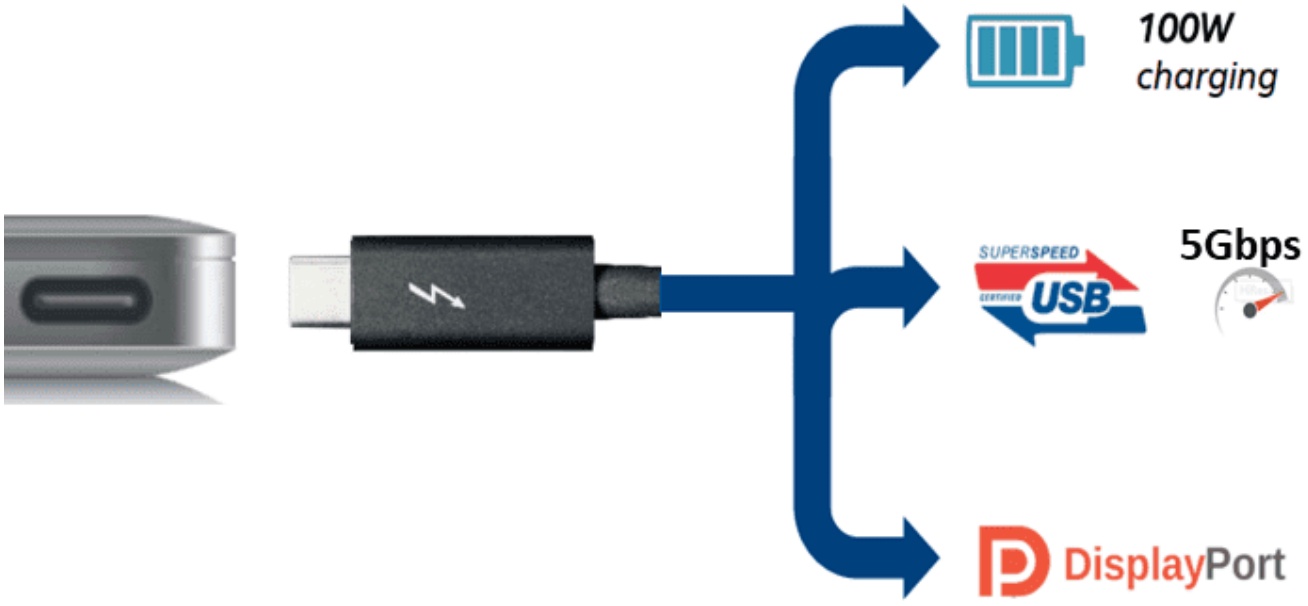

Figure 3 Type-C USB 3.1/DP Interface

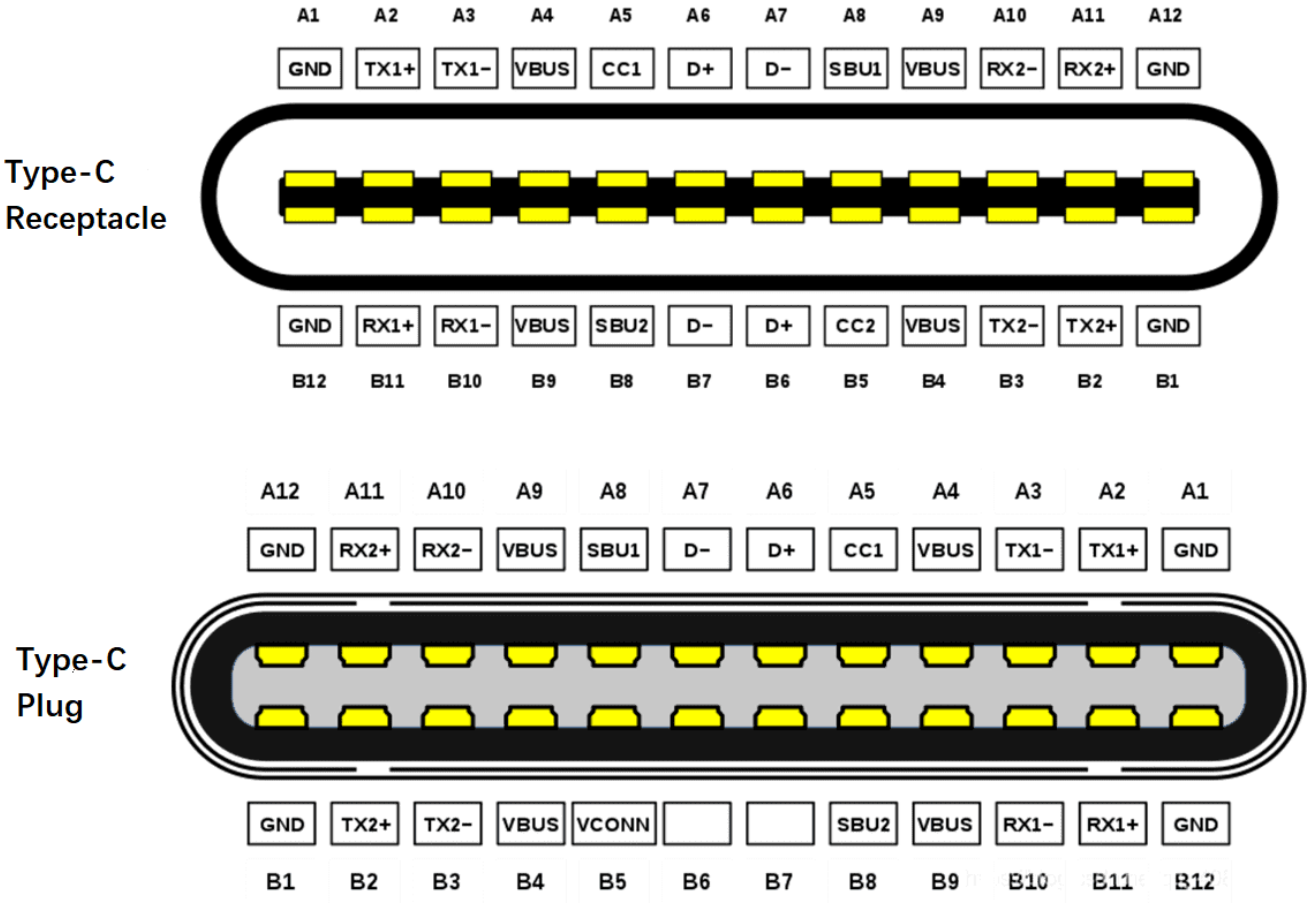

Figure 4 Pin Definition of Type-C Interface

Table 5 Description of Type-C Interface

Pin	Name	Description	Pin	Name	Description
A1	GND	Ground	B12	GND	Ground
A2	SSTXp1	SuperSpeed Differential Signal TX1+	B11	SSRXp1	SuperSpeed Differential Signal RX1+
A3	SSTXn1	SuperSpeed Differential Signal TX1-	B10	SSRXn1	SuperSpeed Differential Signal RX1-
A4	VBUS	USB Bus Power	B9	VBUS	USB Bus Power
A5	CC1	Configuration Channel	B8	SBU2	Sideband Use (SBU)
A6	Dp1	USB 2.0 Differential Signal D1+	B7	Dn2	USB 2.0 Differential Signal D2-
A7	Dn1	USB 2.0 Differential Signal D1-	B6	Dp2	USB 2.0 Differential Signal D2+
A8	SBU1	Sideband Use (SBU)	B5	CC2	Configuration Channel
A9	VBUS	USB Bus Power	B4	VBUS	USB Bus Power
A10	SSRXn2	SuperSpeed Differential Signal RX2-	B3	SSTXn2	SuperSpeed Differential Signal TX2-
A11	SSRXp2	SuperSpeed Differential Signal RX2+	B2	SSTXp2	SuperSpeed Differential Signal TX2+
A12	GND	Ground	B1	GND	Ground

Table 6 Connection Relationship between RK3588 Type-C0 and Type-C Interface

RK3588 Type-C0 Pin	Type-C Interface Pin	Relationship Description
TYPEC0_SSRX1P/DP0_TX0P TYPEC0_SSRX1N/DP0_TX0N	B10/B11	Connected to RK3588 USBDP PHY lane0, can be used for USB 3.1 Rx or DP Tx
TYPEC0_SSTX1P/DP0_TX1P TYPEC0_SSTX1N/DP0_TX1N	A2/A3	Connected to RK3588 USBDP PHY lane1, can be used for USB 3.1 Tx or DP Tx
TYPEC0_SSRX2P/DP0_TX2P TYPEC0_SSRX2N/DP0_TX2N	A10/A11	Connected to RK3588 USBDP PHY lane2, can be used for USB 3.1 Rx or DP Tx
TYPEC0_SSTX2P/DP0_TX3P TYPEC0_SSTX2N/DP0_TX3N	B2/B3	Connected to RK3588 USBDP PHY lane3, can be used for USB 3.1 Tx or DP Tx
TYPEC0_OTG_DP/DM	A6/A7, B6/B7	Connected to RK3588 TYPEC0_OTG_DP/DM, where A6 and B6 are paralleled, A7 and B7 are paralleled.
TYPEC0_SBU1/TYPEC0_SBU2	A8/B8	Connected to RK3588 USBDP PHY AUX, used only for DP Alternate Mode
TYPEC0_SBU1_DC/TYPEC0_SBU2_DC	A8/B8	Connected to RK3588 GPIO, used for software control of pull-up during DP AUX transmission. No requirement for the default pull-up and pull-down method of GPIO
TYPEC0_CC1/TYPEC0_CC2	A5/B5	Connected to the external Type-C controller chip (HUSB311/FUSB302), not connected to R3588 SoC

2.1.2 Type-C to Type-A USB 3.1/DP Interface

The RK3588 Type-C0/1 can be split into separate Type-A USB 3.1 interfaces and DP interfaces for use.

For specific hardware circuit design, please refer to Type-C to Type-A USB 3.1/DP Hardware Circuit.

Taking the Type-C to Type-A USB 3.1/DP interface design of RK3588 EVB2 as an example:

Type-C0: Type-A USB 3.1 (using USBDP PHY0's lane0/1) + DP 1.4 (using USBDP PHY0's lane2/3);

Type-C1: Type-A USB 3.1 (using USBDP PHY1's lane0/1) + DP to VGA (using USBDP PHY1's lane2/3);

Note:

In theory, the hardware can allocate Type-A USB 3.1 to use lane2/3, and DP to use lane0/1, while the software only needs to modify the attribute `rockchip,dp-lane-mux` of the Linux usbdp_phy node for adaptation.

2.1.3 Type-C to Type-A USB 2.0/DP Interface

The RK3588 Type-C0/1 can be split into separate Type-A USB 2.0 interfaces and DP (4 x Lane) interfaces for use.

For specific hardware circuit design, please refer to Type-C to Type-A USB 2.0/DP Hardware Circuit.

Taking the Type-C1 to Type-A USB 2.0/DP interface design of the RK3588 NVR DEMO board as an example:

Type-C1: Type-A USB 2.0 (USBDP PHY not used) + DP 4 x lane to HDMI2.0 (using USBDP PHY1's lane0/1/2/3)

2.1.4 Type-C USB 2.0 Only Interface

RK3588 Type-C0/1 can be simplified to a Type-C USB 2.0 Only Interface. As shown in Figure 5 below:

- Supports Type-C PD (requires an external Type-C Controller Chip)
- Supports USB 2.0 480Mbps data transfer
- Does not support DP Alternate Mode

This design approach is primarily aimed at simplifying the hardware circuit design, but it will reduce the maximum USB transfer rate. Additionally, to accommodate this interface design, significant modifications to the Linux USB DTS are required. Please refer to Type-C USB 2.0 Only DTS Configuration.

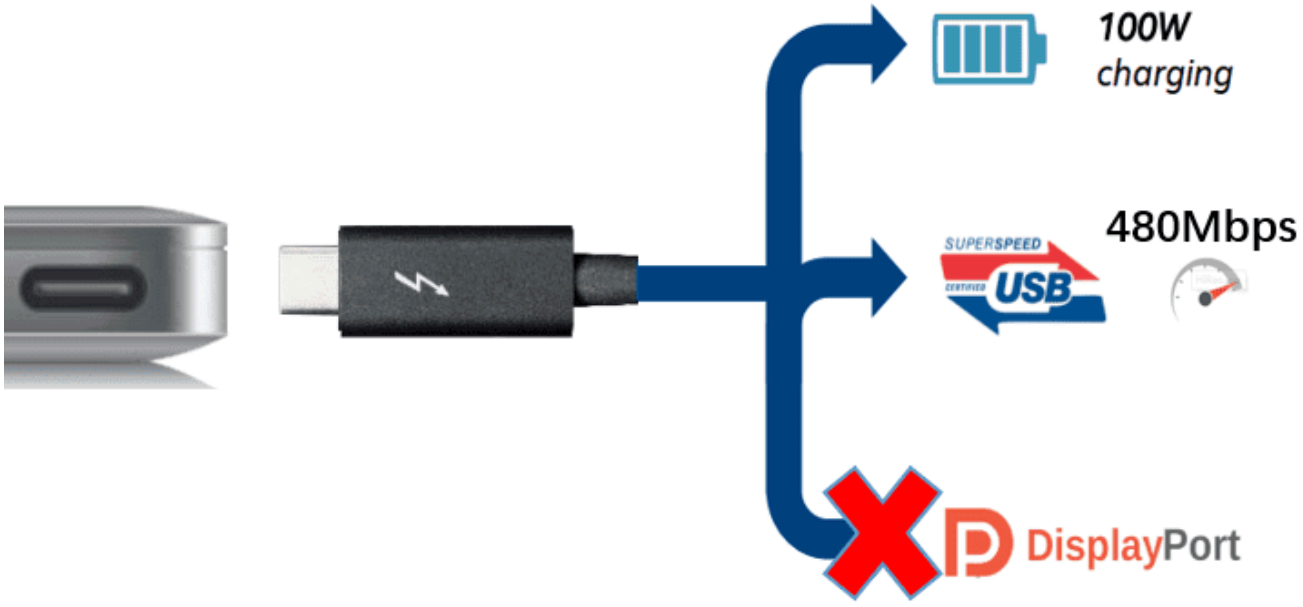

Figure 5 Type-C USB 2.0 Only Interface

2.2 Type-A Interface Type

2.2.1 Type-A USB 3.1 Interface

The RK3588 can support up to 3 Type-A USB 3.1 interfaces, including:

- Type-C0 to Type-A USB 3.1
- Type-C1 to Type-A USB 3.1
- USB3_HOST2 + USB 2.0 HOST0/1

For detailed hardware circuit design, please refer to Type-A USB 3.1 Hardware Circuit.

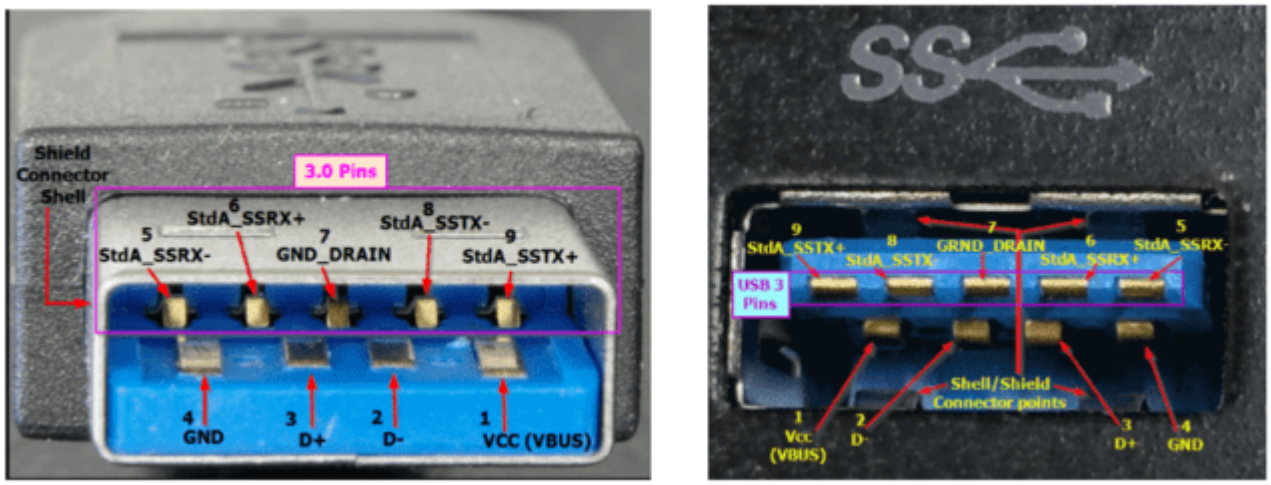

Figure 6 Type-A USB 3.1 Interface

2.2.2 Type-A USB 2.0 Interface

The RK3588 can support up to 4 Type-A USB 2.0 interfaces, including:

- Type-C0 to Type-A USB 2.0
- Type-C1 to Type-A USB 2.0
- USB 2.0 HOST0
- USB 2.0 HOST1

For detailed hardware circuit design, please refer to Type-A USB 2.0 Hardware Circuit.

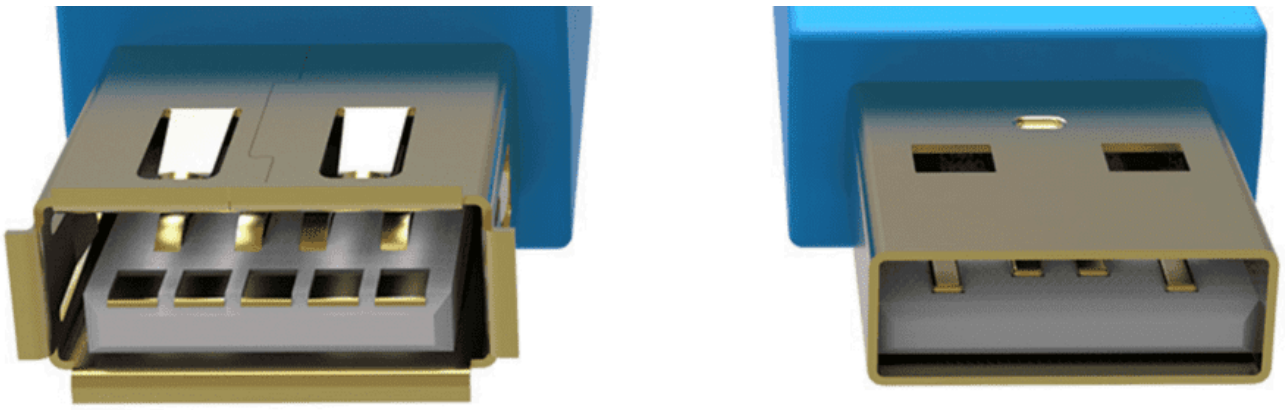

Figure 7 Type-A USB 2.0 Interface

2.3 Micro Interface Types

2.3.1 Micro USB 3.1 Interface

The RK3588 Type-C0/1 can support the design of Micro USB 3.1 interfaces. However, considering the larger PCB area occupied by the Micro USB 3.1 interface, it is less commonly used in current products.

The difference in pins between Micro USB 3.1 and Type-A USB 3.1 is primarily the addition of the OTG ID pin, which is used for hardware automatic detection of ID voltage levels and switching between OTG Device/HOST modes. The OTG ID pin needs to be connected to the RK3588 USB20_OTG_ID pin. Internally, the chip has a default pull-up of ID to a high voltage level of 1.8V, so there is no need for external circuit pull-up.

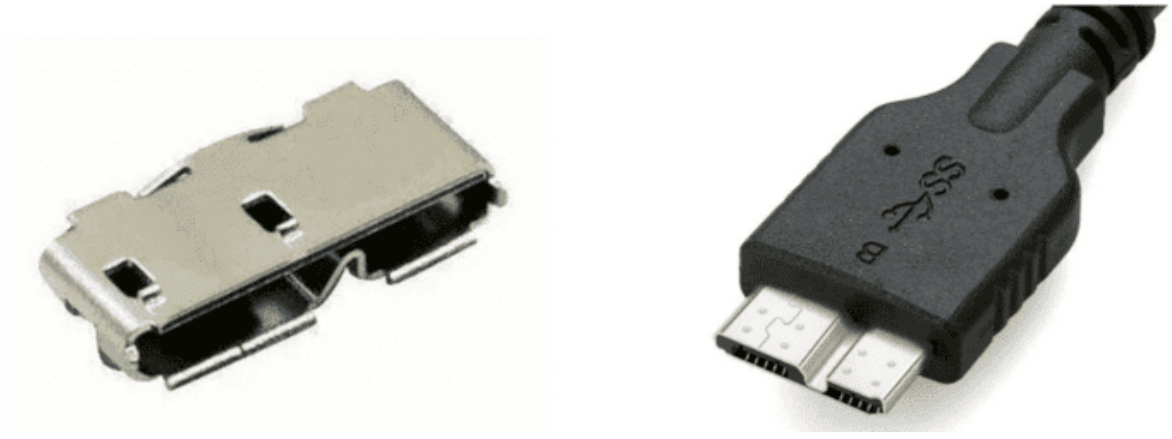

Figure 8 Micro USB 3.1 Interface

2.3.2 Micro USB 2.0 Interface

RK3588 Type-C0 can support the interface design of Micro USB 2.0. The primary purpose of this design approach is to simplify the hardware circuit design, but it will reduce the maximum USB transfer rate.

The pin difference between Micro USB 2.0 and Type-A USB 2.0 mainly involves the addition of the OTG ID pin, which is used for hardware automatic detection of ID voltage levels and switching between OTG Device/HOST modes. The OTG ID pin needs to be connected to the RK3588 USB20_OTG_ID pin. The ID is internally pulled up to a high voltage level of 1.8V within the chip, so no external pull-up is required in the circuit.

Figure 9 Micro USB 2.0 Interface

3. RK3588 USB Config Map

The RK3588 features 5 independent USB controllers and 7 independent USB PHYs, which can support the configuration methods listed in Figure 10 below.

Type-C0/1 can support 4 configurations:

Config0: Type-C0 with DP function

Config1: USB 2.0 OTG + DP 4 x Lane (Swap off)

Config2: USB 2.0 OTG + DP 4 x Lane (Swap on)

Config3: USB 3.1 OTG + DP 2 x Lane (Swap on)

Config4: USB 3.1 OTG + DP 2 x Lane (Swap off)

USB 2.0 HOST0/1 and USB3_HOST2 supported configurations:

Config0: USB 2.0 HOST0 + USB 2.0 HOST1 (USB3_HOST2 not used)

Config1: USB 2.0 HOST0 + USB 2.0 HOST1 Combo with USB3_HOST2

Config2: USB 2.0 HOST0 Combo with USB3_HOST2 + USB 2.0 HOST1

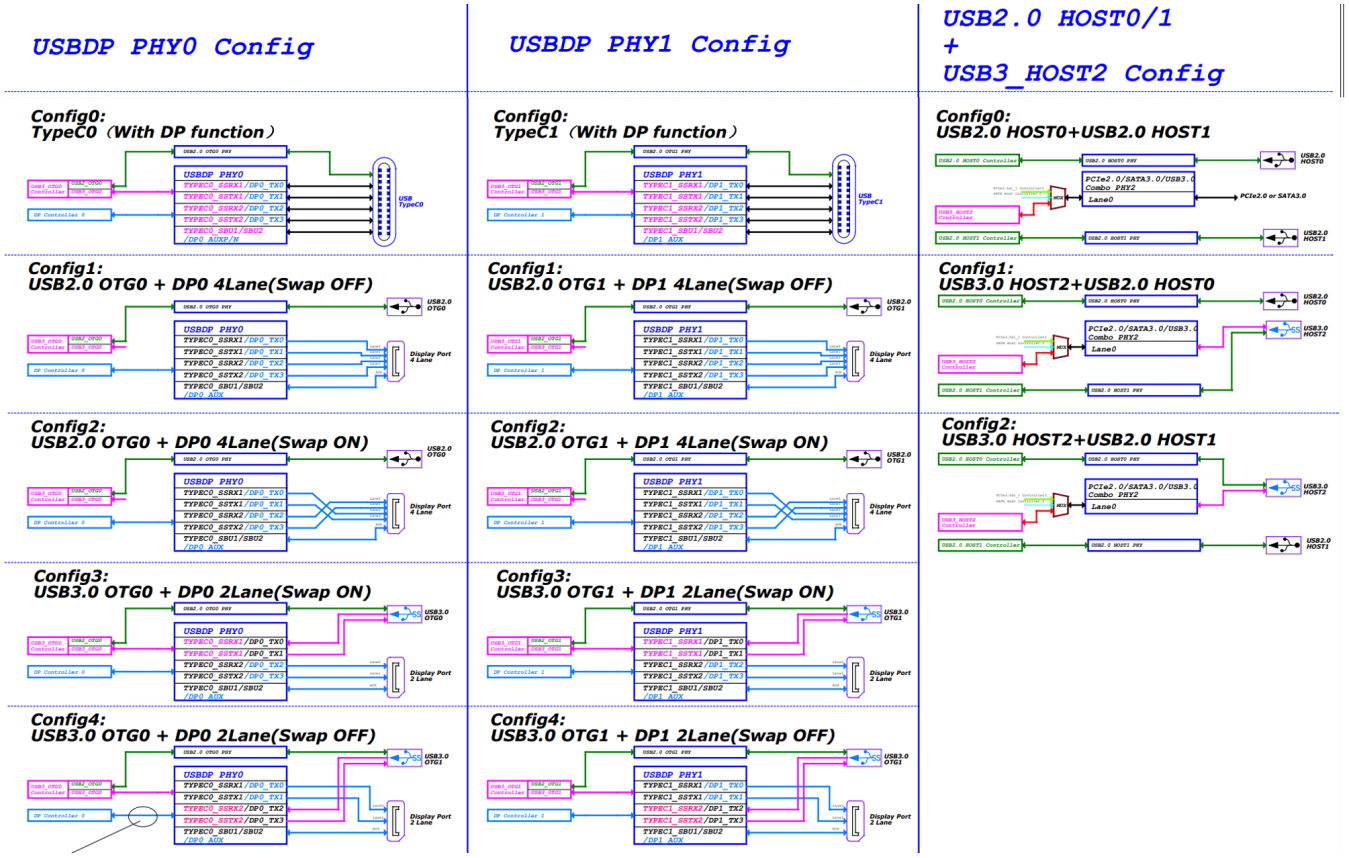

Figure 10 RK3588 USB Config Map

For a more detailed USB configuration table, please refer to the USB Controller Configure Table in the SDK EVB reference schematic.

It should be noted that the DP controller and USBDP PHY's 4 lanes support arbitrary mapping, as shown in Figure 11, by modifying the USBDP PHY's DTS for adaptation. If developers are unsure how to adapt the software, it is recommended to refer to the conventional configurations listed in Figure 10 for hardware circuit design.

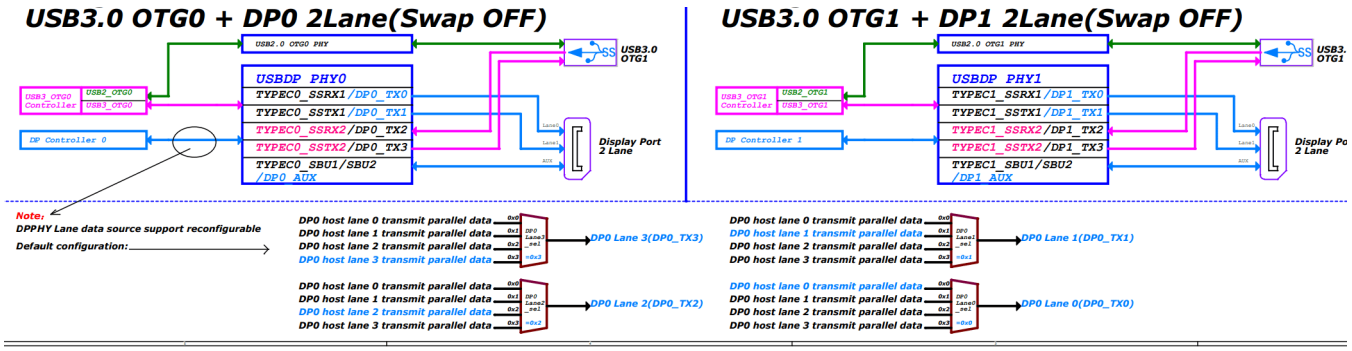

Figure 11 RK3588 DP Lane Map

4. RK3588 USB Hardware Circuit Design

This section primarily outlines the various hardware circuit design solutions that RK3588 USB can support in practical applications. As shown in Figure 12 below, the interfaces supported by RK3588 are as follows:

- USB20 HOST0

- USB20 HOST1
- USB30/DP1.4 MULTI0
- USB30/DP1.4 MULTI1
- USB30/PCIE2.0/SATA30 MULTI2

The RK3588S lacks the USB30/DP1.4 MULTI1 interface.

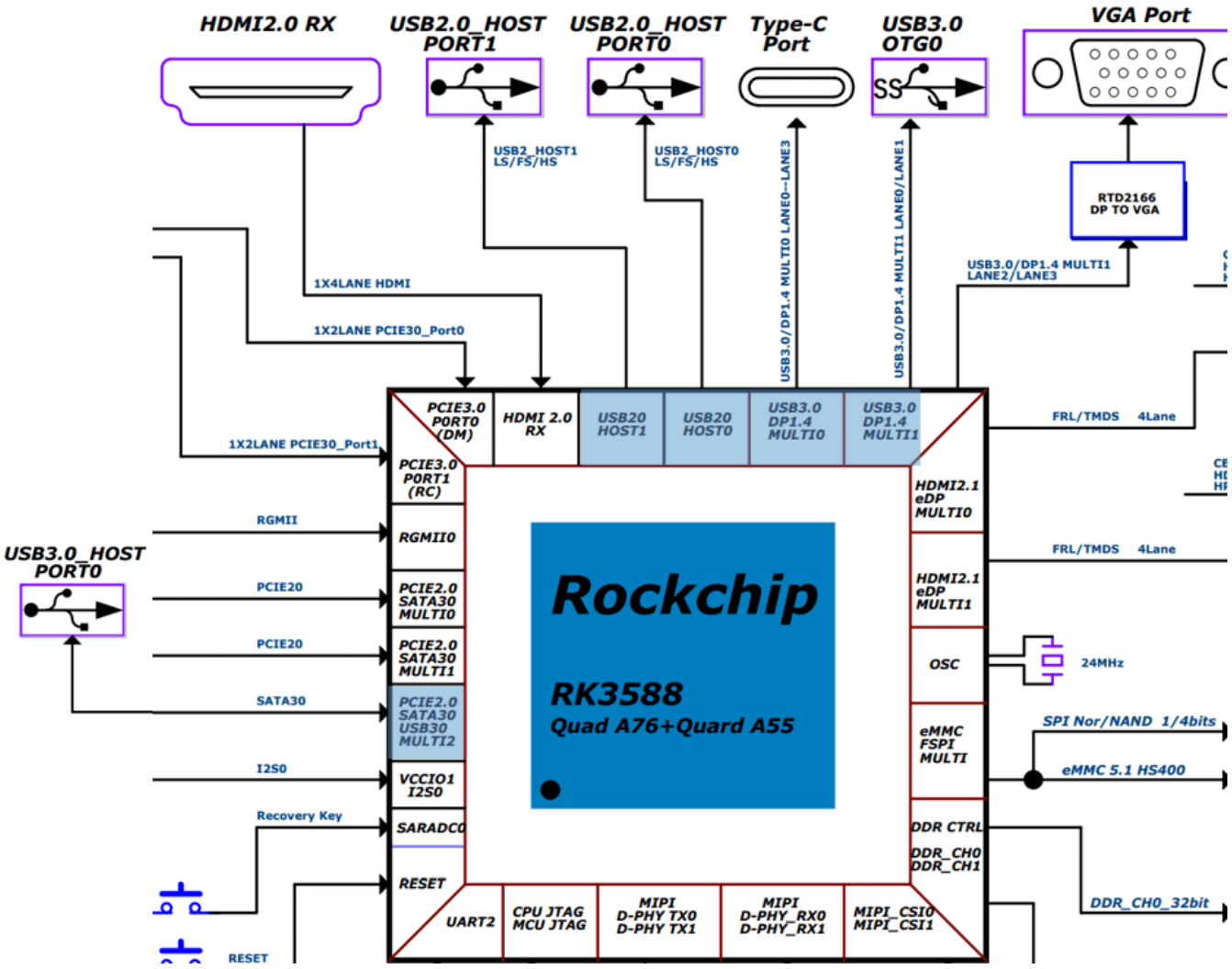

Figure 12 RK3588 USB Interface Block Diagram

4.1 USB Controller Power Supply and Power Consumption Management

The power supply for the RK3588 USB controller is VDD_LOGIC. Additionally, the chip has an internally designed power domain specifically for USB controllers, with the corresponding Power Domain for each USB controller as shown in Table 7 below.

Table 7: Correspondence between USB Controllers and PD

USB Controller	Power Domain
USB 2.0 HOST0/1	PD_USB
USB 3.1 OTG0/1	PD_USB
USB 3.1 HOST2	PD_PHP

In practical application scenarios, the Linux USB controller driver will dynamically switch the PD of the USB controller based on the working condition of the USB interface, using the Linux PM Runtime mechanism to reduce the power consumption of the USB controller. When the system enters the second-level standby, in order to achieve the optimal power consumption, the software will forcibly shut down all PDs of the USB controller. Therefore, in actual product application scenarios, if it is necessary to maintain the working state of the USB controller's registers during the second-level standby, the function `device_init_wakeup` needs to be called in the USB controller driver to prevent the PD of the USB controller from being shut down during the second-level standby.

The power consumption control strategy for the USB controller is as follows:

1. For USB controllers that are not used, the corresponding controller DTS node needs to be configured as disabled;
2. For USB controllers enabled by the kernel, the kernel USB driver already supports the Auto suspend feature of the USB controller (when the USB HOST interface is not connected to any external devices, the controller automatically enters a suspend low-power state), so developers do not need to debug the dynamic power management of the USB controller.

The method to disable USB 2.0 HOST0/1 in the kernel is as follows:

```
# Disable USB 2.0 HOST0
&usb_host0_ehci {
        status = "disabled";
};

&usb_host0_ohci {
        status = "disabled";
};

# Disable USB 2.0 HOST1
&usb_host1_ehci {
        status = "disabled";
};

&usb_host1_ohci {
        status = "disabled";
};
```

4.2 USB PHY Power Supply and Power Consumption Management

4.2.1 USB 2.0 PHY Power Supply and Power Consumption Management

The RK3588 supports four independent USB 2.0 PHYs. The RK3588S has one less USB 2.0 PHY1 compared to the RK3588. Internally, all USB 2.0 PHYs belong to the PD_BUS (Alive), and all USB 2.0 PHYs share the three power supply lines as shown in Figure 13 below. Therefore, during system operation, it is not possible to reduce the power consumption of the USB 2.0 PHYs by simply turning off the hardware power and closing PD.

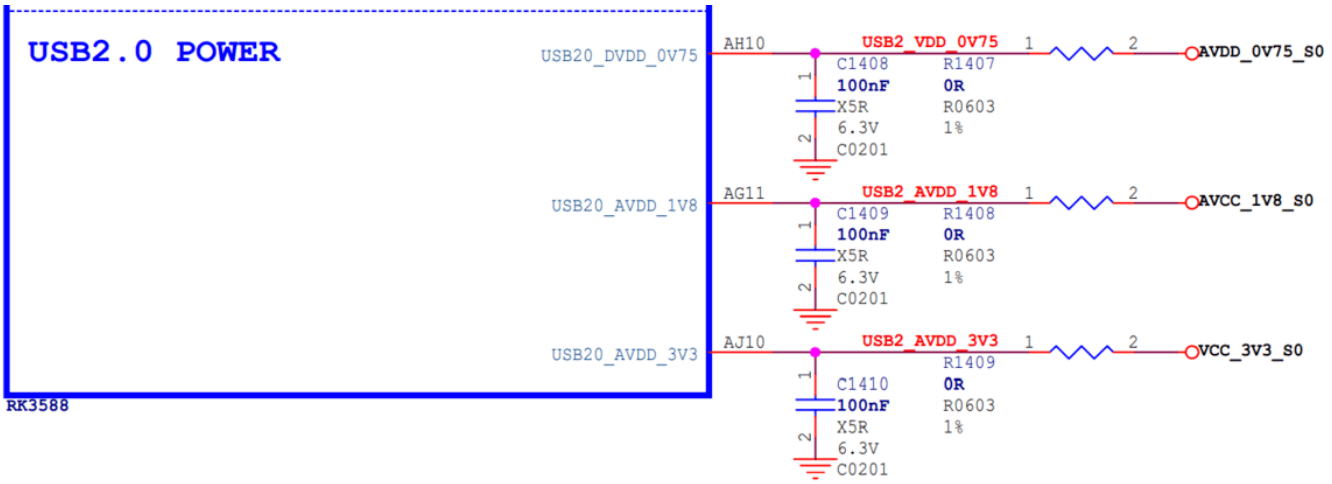

Figure 13 USB 2.0 PHY Power Supply

It should be noted that in actual circuits, if the supply voltage of the USB 2.0 PHY exceeds the specified maximum value or is below the specified minimum value, it may cause USB connection anomalies.

Table 8 USB 2.0 PHY Voltage Supply Requirements

Power Supply	Minimum	Normal	Maximum	Unit
USB20_DVDD_0V75	0.6975	0.75	0.825	V
USB20_AVDD_1V8	1.674	1.8	1.98	V
USB20_AVDD_3V3	3.069	3.3	3.63	V

USB 2.0 PHY Power Consumption Control Strategy is as follows:

1. To support the function of Maskrom USB downloading firmware, it is necessary to ensure that the power supply of the USB 2.0 PHY is normal;
2. After the system is powered on, all USB 2.0 PHYs are in Normal mode by default. The software configures USB 2.0 PHY1/PHY2/PHY3 in the lowest power consumption IDDQ mode during the U-Boot SPL phase (SDK is already supported). After entering the system, the kernel USB driver will set the corresponding USB 2.0 PHY to exit IDDQ mode according to the application requirements;
3. For unused USB 2.0 PHYs, the corresponding USB 2.0 PHY DTS nodes need to be configured as disabled (refer to Table 9);
4. For the USB 2.0 PHYs enabled by the kernel, the kernel USB 2.0 PHY driver will automatically perform dynamic power consumption control on the PHY. When a device is detected, it automatically sets the

USB 2.0 PHY to Normal mode. When no device is detected, it automatically sets the USB 2.0 PHY to Suspend mode;

The power consumption data of the USB 2.0 PHY in different working modes is shown in Table 9 below.

Table 9 USB 2.0 PHY Power Consumption Data

(Statistics for a single USB 2.0 PHY)

Power Supply	Read/Write Data	Dynamic Suspend	PHY Disabled	Second-Level Standby	Unit
USB20_DVDD_0V75	7.1	2.6	0.05	0	mA
USB20_AVDD_1V8	17.8	3.1	0.05	0	mA
USB20_AVDD_3V3	3.3	0.05	0.05	0	mA

Note:

- The test scenario for read/write data power consumption: Copying data with a U2 disk, PHY is in Normal mode;
- The test scenario for dynamic suspend power consumption: USB 2.0 PHY DTS is enabled, but no USB peripheral is connected, PHY is in Suspend mode;
- The test scenario for PHY disabled power consumption: USB 2.0 PHY DTS is disabled, PHY is in IDDQ mode;
- The test scenario for second-level standby power consumption: All three power supply lines of the USB 2.0 PHY are turned off;

Table 10 Connection Relationship between USB 2.0 PHY and USB Controller

USB 2.0 PHY	USB Controller
USB 2.0 PHY0	USB 3.1 OTG0
USB 2.0 PHY1	USB 3.1 OTG1
USB 2.0 PHY2	USB 2.0 HOST0
USB 2.0 PHY3	USB 2.0 HOST1

Method to disable USB 2.0 PHY2/3 in the kernel is as follows:

```
&u2phy2 {
        status = "disabled";
};

&u2phy3 {
        status = "disabled";
};

&u2phy2_host {
```

```
        status = "disabled";
};

&u2phy3_host {
        status = "disabled";
};
```

4.2.2 USB 3.1 PHY Power Supply and Power Consumption Management

RK3588 supports two types of USB 3.1 Combo PHYs:

1. USB 3.1/DP Combo PHY
2. USB 3.1/SATA/PCIe Combo PHY

Table 11 Connection Relationship between USB 3.1 Combo PHY and USB Controller

USB 3.1 Combo PHY	USB Controller
USB 3.1/DP Combo PHY0	USB 3.1 OTG0
USB 3.1/DP Combo PHY1	USB 3.1 OTG1
USB 3.1/SATA/PCIe Combo PHY2	USB 3.1 HOST2

The power supply and power consumption control methods for these two types of USB 3.1 Combo PHYs are different and are described separately below.

4.2.2.1 USB 3.1/DP Combo PHY

The RK3588 supports two independent USB 3.1/DP Combo PHYs. The RK3588S has one less USB 3.1/DP Combo PHY1 compared to the RK3588. Internally, both USB 3.1/DP Combo PHYs belong to the PD_BUS (Alive), and externally, each PHY has its own independent power supply pin, as shown in Figure 14.

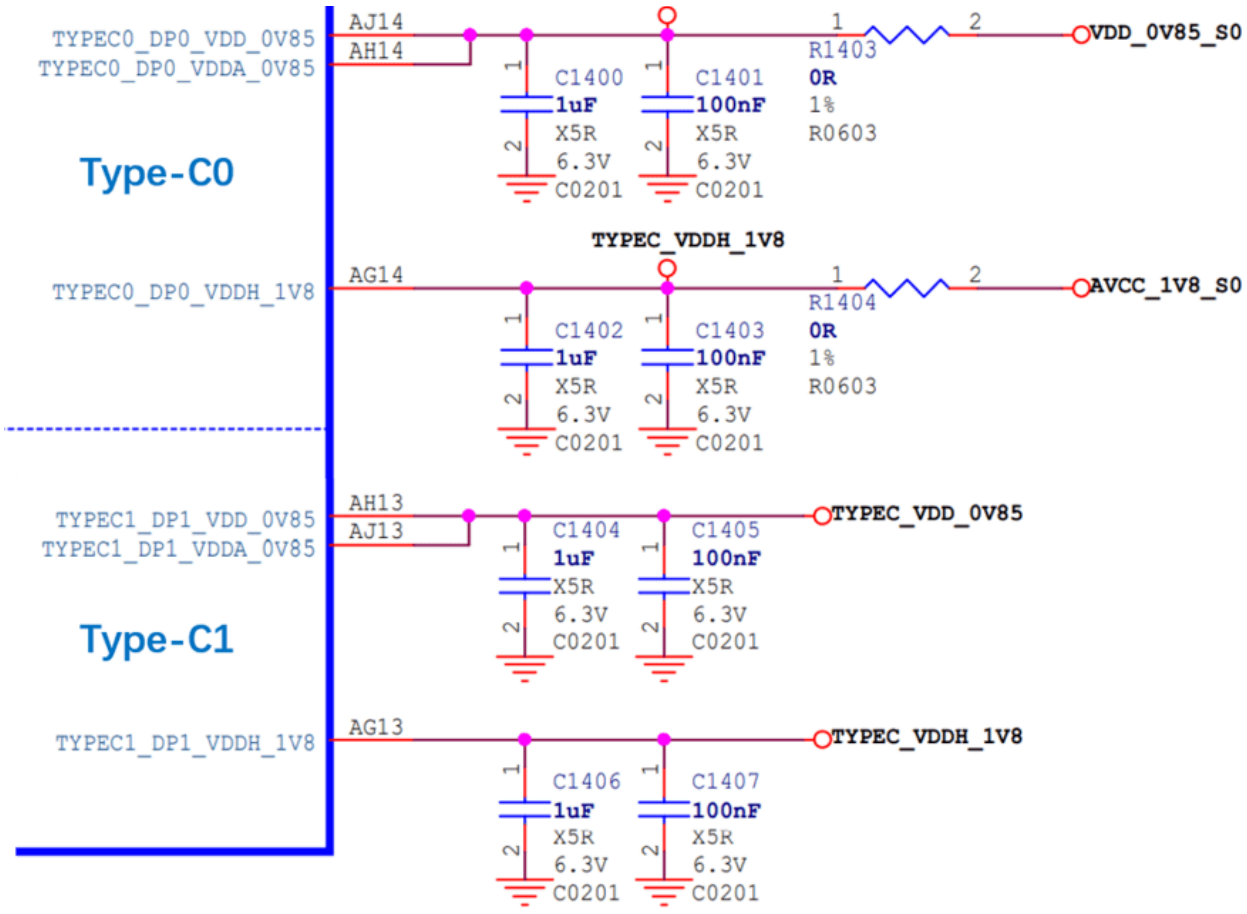

Figure 14 RK3588 USB 3.1/DP Combo PHY Power Supply

Table 12 USB 3.1/DP Combo PHY Power Voltage Requirements

Power Supply	Minimum	Nominal	Maximum	Unit
VDD_0V85/VDDA_0V85	0.8075	0.85	0.8925	V
VDDH_1V8	1.71	1.8	1.89	V

USB 3.1/DP Combo PHY Power Consumption Control Strategy is as follows:

1. **To support the Maskrom USB firmware download function, it is necessary to ensure that the power supply of Type-C0 USBDP PHY0 is normal**;
2. When the Type-C1 interface is not in use, the corresponding Type-C1 USBDP PHY1 can be powered off;
3. After the system is powered up, the power consumption of USBDP PHY in the uninitialized state is the lowest;
4. For unused USBDP PHYs, the corresponding USBDP PHY DTS node should be configured as disabled, which means keeping the PHY in the uninitialized state with the lowest power consumption;
5. For the USBDP PHYs enabled by the kernel, the kernel USBDP PHY driver will automatically perform dynamic power consumption control on the PHY. When a device is detected, it automatically sets the USBDP PHY to P0 State. When no device is detected, it automatically sets the USBDP PHY to P3 State (for Type-A interfaces) or reset state (for Type-C interfaces);

Table 13 USB 3.1/DP Combo PHY Power Consumption Data

(Statistics for a single USB 3.1/DP Combo PHY's power consumption)

Power Supply	Read/Write Data	Dynamic Sleep[1]	Dynamic Sleep[2]	PHY Disabled	Deep Standby	Unit
VDD_0V85/VDDA_0V85	115.7	6.8	7.9	2	0	mA
VDDH_1V8	29.4	0	2	0	0	mA

Note:

- The power consumption test scenario for read/write data: Copying data from a U3 disk, PHY is in P0 state;
- The power consumption test scenario for dynamic sleep power[1]: Type-C interface, no USB peripheral connected, PHY is in reset state;
- The power consumption test scenario for dynamic sleep power[2]: Type-A interface, no USB peripheral connected, PHY is in P3 state;
- The power consumption test scenario for PHY disabled: The DTS node of PHY is configured as disabled, PHY is in the uninitialized state with the lowest power consumption;
- The power consumption test scenario for deep standby: Both power supplies of PHY are turned off;

Method to disable USBDP PHY1 in the kernel is as follows:

```
&usbdp_phy1 {
        status = "disabled";
};

&usbdp_phy1_dp {
        status = "disabled";
};

&usbdp_phy1_u3 {
        status = "disabled";
};
```

4.2.2.2 USB 3.1/SATA/PCIe Combo PHY

RK3588 supports 1 USB3.1/SATA/PCIe Combo PHY. Internally within the chip, this PHY belongs to PD_BUS (Alive), and externally, the PHY has an independent power supply as shown in Figure 15.

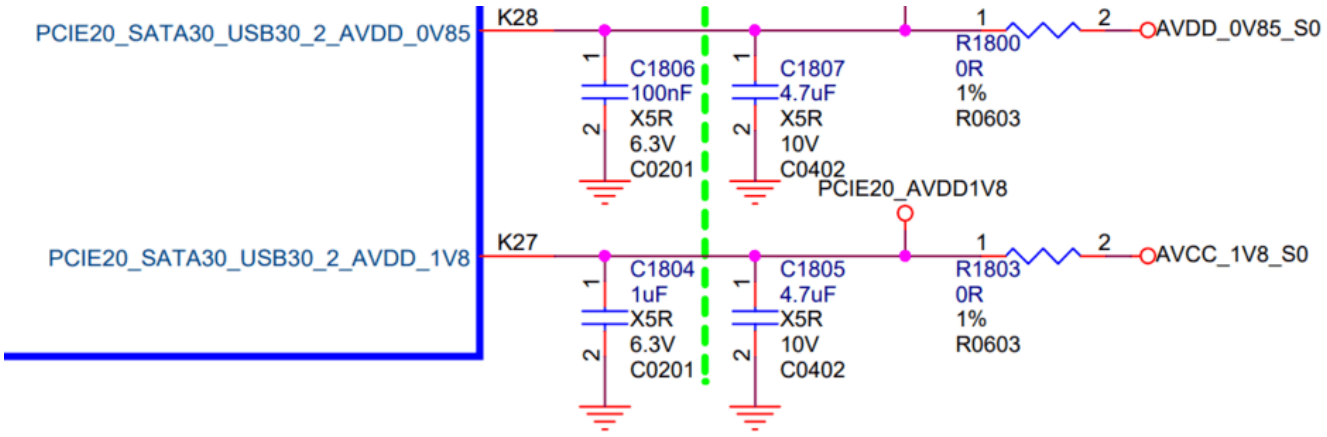

Figure 15 RK3588 USB 3.1/SATA/PCIe Combo PHY Power Supply

Table 14 USB 3.1/SATA/PCIe Combo PHY Power Supply Voltage Requirements

Power Supply	Minimum	Normal	Maximum	Unit
AVDD_0V85	0.8	0.85	0.935	V
AVDD_1V8	1.62	1.8	1.98	V

USB 3.1/SATA/PCIe Combo PHY Power Consumption Control Strategy is as follows:

1. When the chip is powered on, the USB 3.1/SATA/PCIe Combo PHY defaults to the working state. Software configures the PHY to be in reset state during the U-Boot SPL phase to maintain the lowest power consumption of the PHY. After entering the kernel, the USB controller driver releases the PHY's reset by calling the `rockchip_combphy_init()` function.
2. Dynamic power consumption control of the PHY: When the Combo PHY operates in USB mode, the PIPE state (P0/P1/P2/P3) of the PHY is automatically controlled by the USB controller hardware, dynamically entering and exiting P0/P1/P2/P3 states according to different working scenarios. For example, when no USB device is inserted, the PIPE is in P3 state; when a U3 disk is inserted, it switches to P0 state; when a U3 HUB is connected, as long as the downstream port of the HUB is not connected to other USB peripherals, the PIPE state automatically enters the P3 state low power. When a USB peripheral is inserted into the U3 HUB, the PIPE state automatically switches to P0. (Note: P0 is the normal working state, P3 is the lowest power state)
3. When it is clear that the USB 3.1 HOST2 interface is not used, the corresponding USB 3.1/SATA/PCIe Combo PHY can be unpowered;
4. When the PHY is powered, if this PHY is not used, the corresponding PHY DTS node needs to be configured as disabled, which means keeping the PHY in reset state with the lowest power consumption;

Table 15 USB 3.1/SATA/PCIe Combo PHY Power Consumption Data

Power Supply	Read/Write Data	Dynamic Sleep	PHY Disabled	Second-Level Standby	Unit
AVDD_0V85	43.4	43.3	0.4	0	mA
AVDD_1V8	4.3	4.1	0.2	0	mA

Note:

- The test scenario for read/write data power consumption: Connecting a U3 disk to copy data, with PHY in P0 state.
- The test scenario for dynamic sleep: PHY DTS is enabled, but no USB peripheral is connected, with PHY in P3 State;
- The test scenario for PHY Disabled: The DTS node of PHY is configured as disabled, with PHY in reset state;
- The test scenario for second-level standby power consumption: Both power supplies of PHY are turned off;

Method to disable USB 3.1/SATA/PCIe Combo PHY in the kernel is as follows:

```
&combphy2_psu {
        status = "disabled";
};
```

4.3 USB Hardware Circuit Design

4.3.1 TYPEC0_USB20_VBUSDET Circuit Design

TYPEC0_USB20_VBUSDET is used for USB Device scenarios, detecting the connection and disconnection of USB Devices. The design considerations for TYPEC0_USB20_VBUSDET are as follows:

- When Type-C0 is designed to support PD functionality as a Type-C interface, i.e., supporting an external Type-C controller chip (such as: FUSB302 or HUSB311), refer to the circuit design of RK3588 EVB1 Type-C0 (TYPEC0_USB20_VBUSDET is fixedly pulled up to VCC_3V3_S0), the software driver can detect the connection and disconnection of USB Devices through the CC of the Type-C controller chip;
- For other circuit design schemes that do not support an external Type-C controller chip (such as Type-C0 USB 2.0 only, Type-A USB 3.1, Micro USB 2.0/3.1), it is required that TYPEC0_USB20_VBUSDET still follows the traditional voltage-dividing circuit design, connected to the VBUS pin of the USB interface, and VBUSDET is not designed as a constant power supply (if there is a need for USB HOST, an independent GPIO or PMIC VBUS control is required, not shared with other USB HOST interfaces);
- It is required that the high-level range of the chip input end TYPEC0_USB20_VBUSDET is **[0.9V ~ 3.3V]**.

4.3.2 Maskrom USB Circuit Design

The RK3588 Maskrom consistently utilizes Type-C0 USB 2.0 for firmware downloading functionality. The related circuit design is shown in Figure 16 below. To ensure the normal operation of the Maskrom USB downloading function, the following requirements must be met:

- TYPEC0_USB20_VBUSDET must support external pull-up and should not be left floating;
- The power supply for the TypeC0 USBDP PHY (TYPEC0_DP0_VDD_0V85/TYPEC0_DP0_VDDA_0V85/TYPEC0_DP0_VDDH_1V8) and the external reference

resistor (TYPEC0_DP0_REXT) must be properly connected. After entering the system, the software will perform low-power processing on the TypeC0 USBDP PHY.

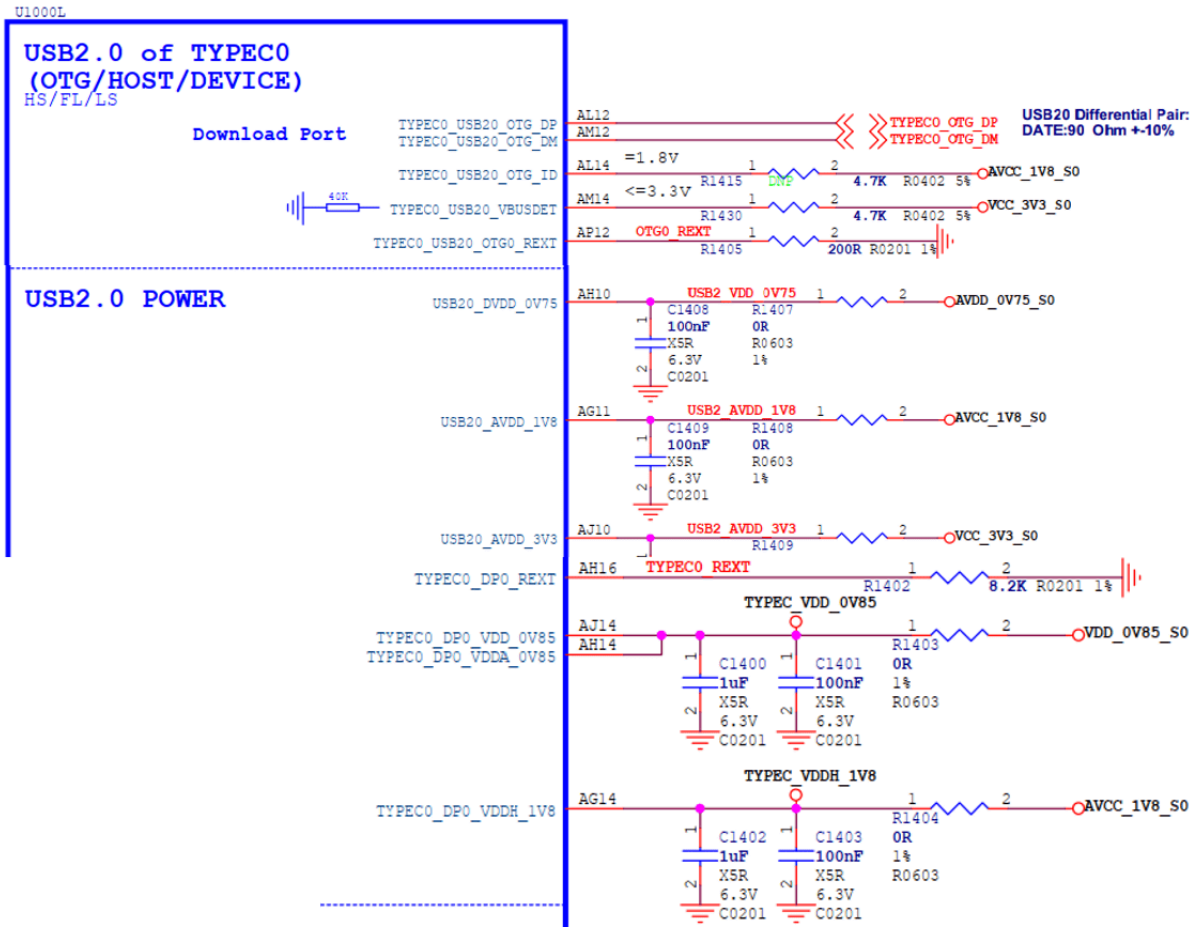

Figure 16 RK3588 Maskrom USB2 Firmware Download Port Circuit

4.3.3 Type-C USB 3.1/DP Full-Function Hardware Circuit

This solution is applicable to RK3588 Type-C0 and Type-C1.

Taking the hardware circuit design of RK3588 EVB1 Type-C0 as an example.

1. For the detailed connection relationship between each Pin of RK3588 Type-C0 and the Type-C interface, please refer to the table 5 in Type-C USB 3.1/DP Full-Function Interface;
2. The Type-C circuit requires an external Type-C controller chip to achieve the full functionality of Type-C (including: detection of both sides, high voltage and high current PD protocol interaction, etc.). RK3588 can support most commonly used Type-C controller chips, for details please refer to Type-C Controller Chip Support List;
3. To support high voltage charging functions and reduce the risk of hardware circuits, TYPEC0_USB20_VBUSDET should not be connected to the VBUS of the Type-C interface, fixed pull-up to 3.3V is sufficient (as shown in Figure 17 TYPEC0_USB20_VBUSDET connected to VCC_3V3_S0), but it must not be left floating;
4. TYPEC0_USB20_OTG_ID is only used for OTG functions of Micro interface types, it is not needed for Type-C circuits, leave it floating.
5. TYPEC0_SBU1/TYPEC0_SBU2 is only used for AUX communication in DP Alternate Mode. According to

the AUX protocol requirements, SBU1/SBU2 should be pulled up to the corresponding voltage level based on the insertion of the Type-C interface's front and back. Since the RK3588 chip does not implement automatic pull-up for SBU1/SBU2 internally, it is required to add two GPIO controls externally in the hardware circuit (corresponding to TYPEC0_SBU1_DC/TYPEC0_SBU2_DC in Figure 19). There are no requirements for the default pull-up and pull-down methods of GPIO, any GPIO can be chosen. In software, it is necessary to modify the properties `sbu1-dc-gpios` and `sbu2-dc-gpios` of the usbdp_phy node for adaptation;

6. For the software configuration of Type-C USB DTS, please refer to Type-C USB 3.1/DP Full-Function DTS Configuration;

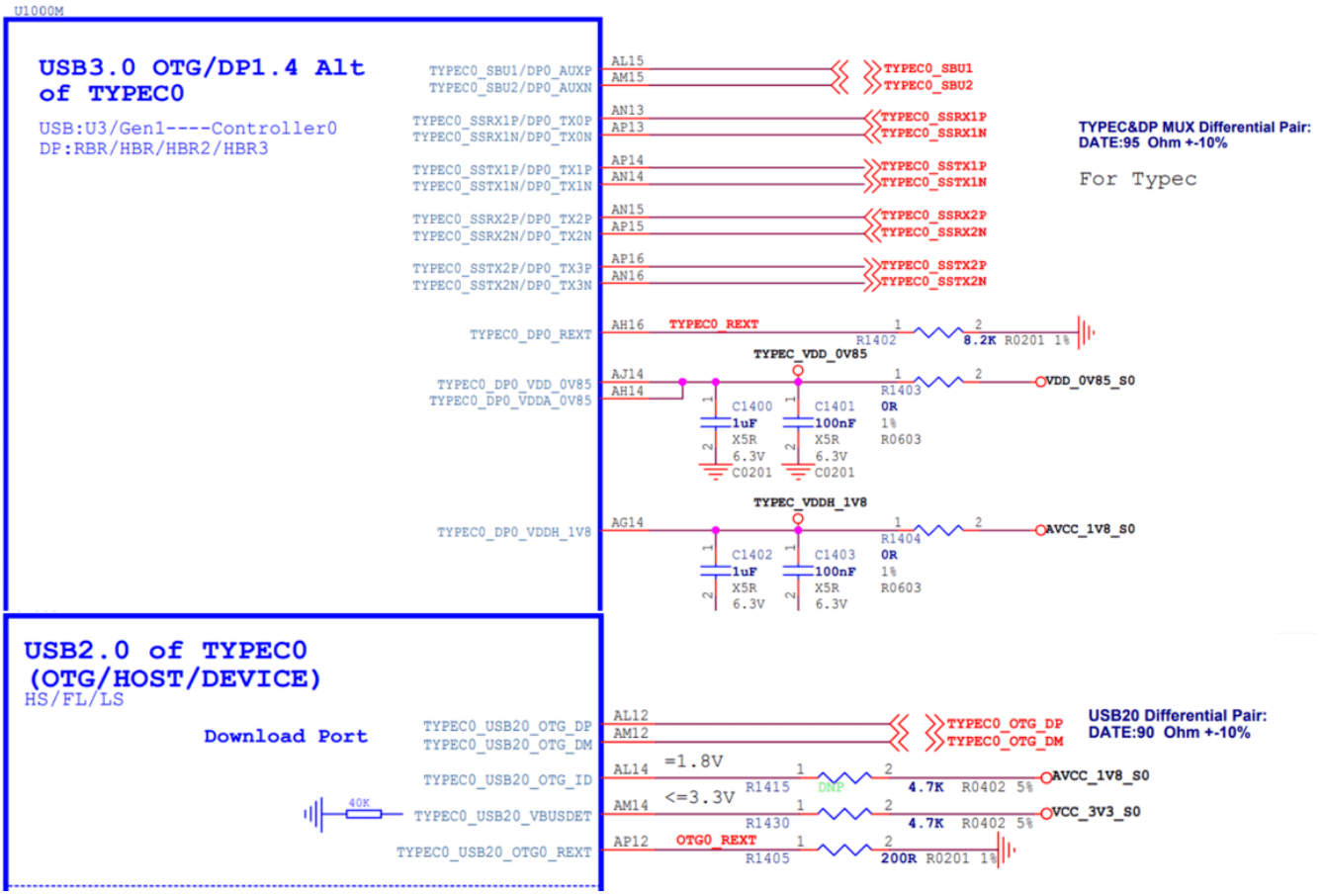

Figure 17 RK3588 Type-C0 Circuit

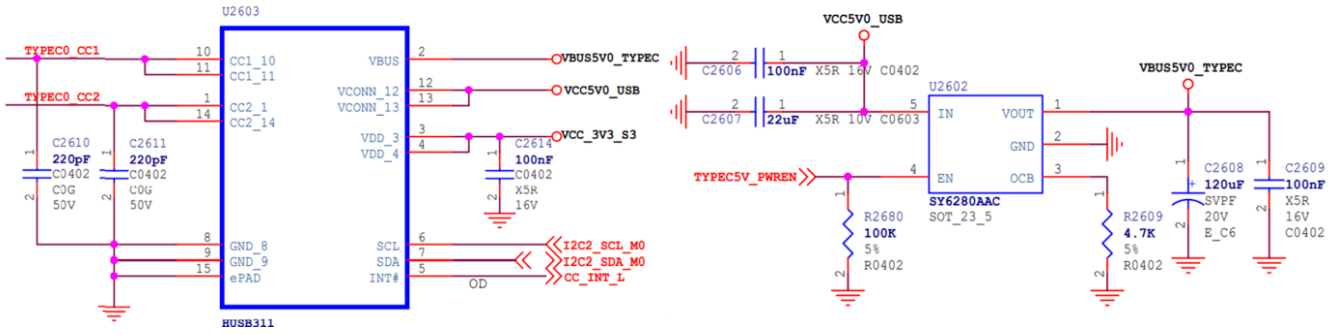

Figure 18 RK3588 Type-C0 PD Controller Circuit and VBUS Control Circuit

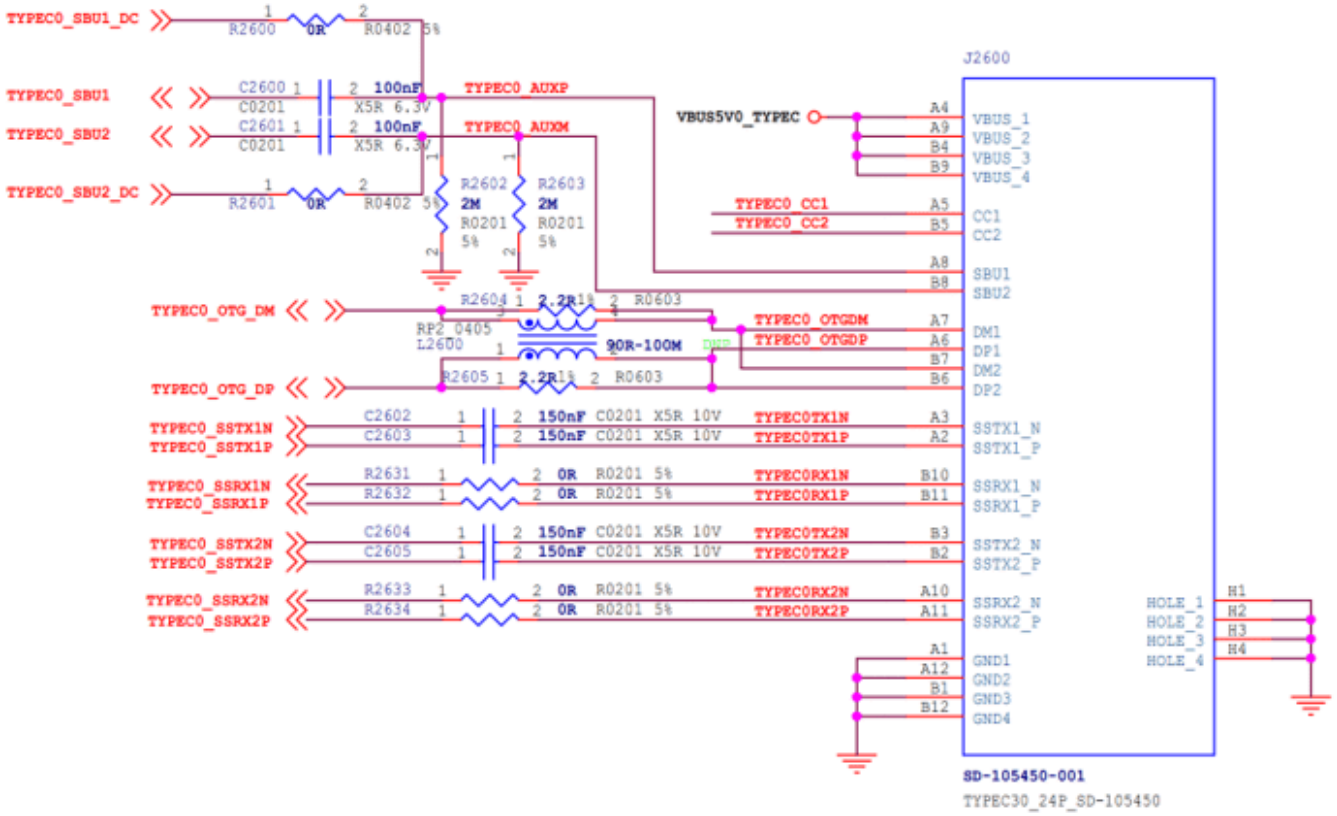

Figure 19 RK3588 Type-C0 Interface

4.3.4 Type-C to Type-A USB 3.1/DP Hardware Circuit

This solution is applicable to RK3588 Type-C0 and Type-C1, and can be split into independent Type-A USB 3.1 interfaces and DP (2 x Lane) interfaces for use.

Taking the RK3588 EVB2 Type-C0 to Type-A USB 3.1/DP hardware circuit design as an example.

1. Type-A USB 3.1 uses TYPEC0_SSRX1P/N and TYPEC0_SSTX1P/N (corresponding to the lane0/1 of the chip's internal USBDP PHY), while DP uses DP0_TX2P/N and DP0_TX3P/N (corresponding to the chip's internal USBDP PHY's lane2/3);
2. If the USB OTG has scenarios for Device/HOST applications, it is recommended that TYPEC0_OTG_VBUSDET be connected to the VBUS of the Type-A USB interface through a 30KΩ resistor;
3. The power supply for Type-A VBUS (VCC5V0_USB30_HOST2) is controlled by GPIO, turning off VBUS output when OTG is in Device mode. When OTG is in HOST mode, VBUS output is turned on. In addition, the output current of the voltage regulator chip SY6280AAC is determined by the resistor connected to the OCB pin, with the maximum current Ilim(A) = 6800/Rset(ohm), as shown in Figure 21 below, and the VBUS output current limit is 1A;
4. For the corresponding DTS configuration of Type-C to Type-A USB 3.1/DP, please refer to Type-C to Type-A USB 3.1/DP DTS Configuration.

Note:

In theory, the hardware circuit can also be designed to use lane2/3 for Type-A USB 3.1 and lane0/1 for DP, but the software needs to modify the attribute `rockchip,dp-lane-mux` of the usbdp_phy node to adapt to the hardware design.

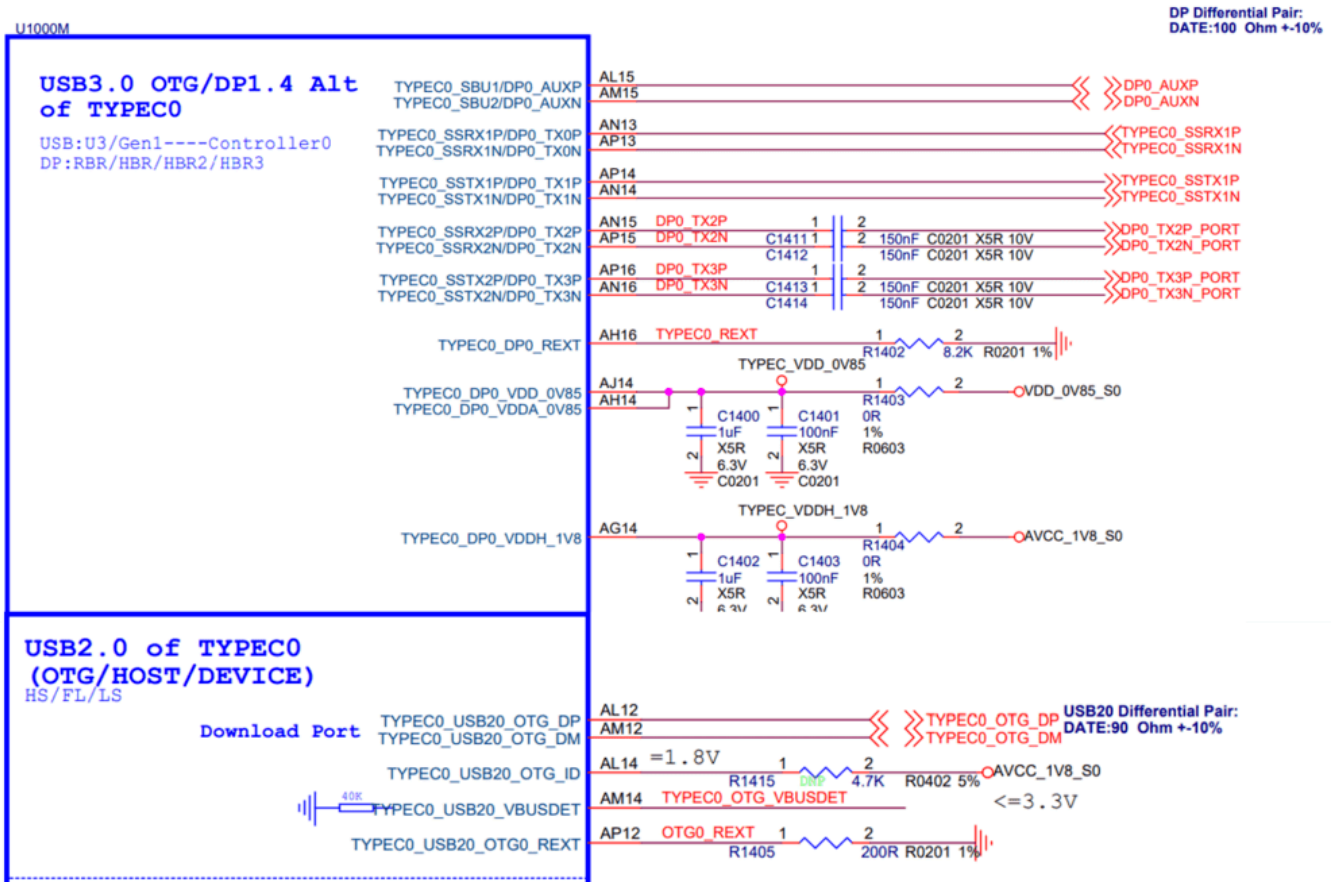

Figure 20 RK3588 Type-A USB 3.1/DP Circuit

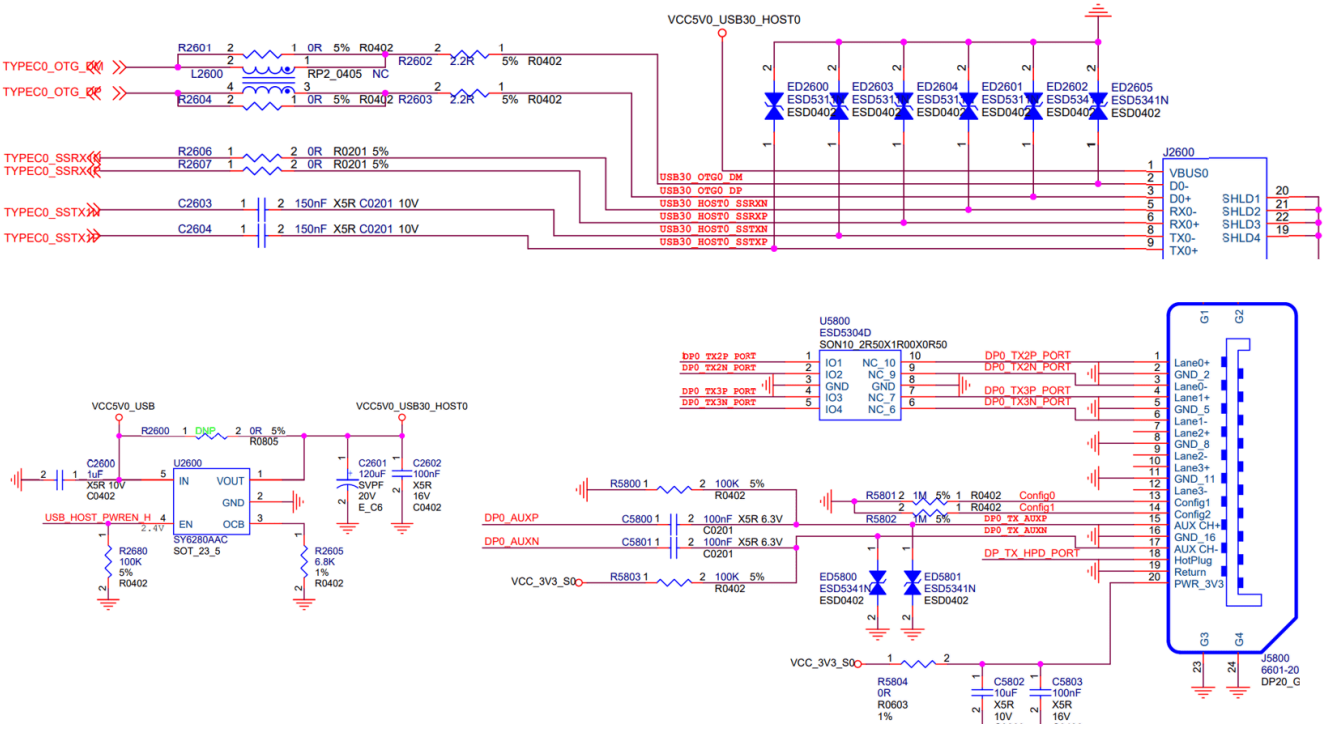

Figure 21 RK3588 Type-A USB3.1/DP Interface

4.3.5 Type-C to Type-A USB 2.0/DP Hardware Circuit

This solution is applicable to RK3588 Type-C0 and Type-C1, which can be separated into independent Type-A USB 2.0 interfaces and DP (4 x Lane) interfaces for use.

Taking the RK3588 NVR Demo Board Type-C1 to Type-A USB 2.0/DP hardware circuit design as an example.

1. The 4 x Lane of TYPEC1 USBDP PHY is fully utilized for the DP interface, and USB does not use the USBDP PHY;
2. When TYPEC1 USB is used only in HOST mode, TYPEC1_USB20_OTG_ID and TYPEC1_USB20_VBUSDET can be left floating;
3. The power supply for Type-A VBUS (VCC5V0_USB20_HOST) is controlled by GPIO, and when OTG is in HOST mode, VBUS output is turned on. Additionally, the output current of the voltage regulator chip SY6280AAC is determined by the resistor connected to the OCB pin, with the maximum current Ilim(A) = 6800/Rset(ohm), as shown in Figure 23 below, and the VBUS output current limit is 1.45A;
4. For the corresponding DTS configuration of Type-C to Type-A USB 2.0/DP, please refer to Type-C to Type-A USB 2.0/DP DTS Configuration.

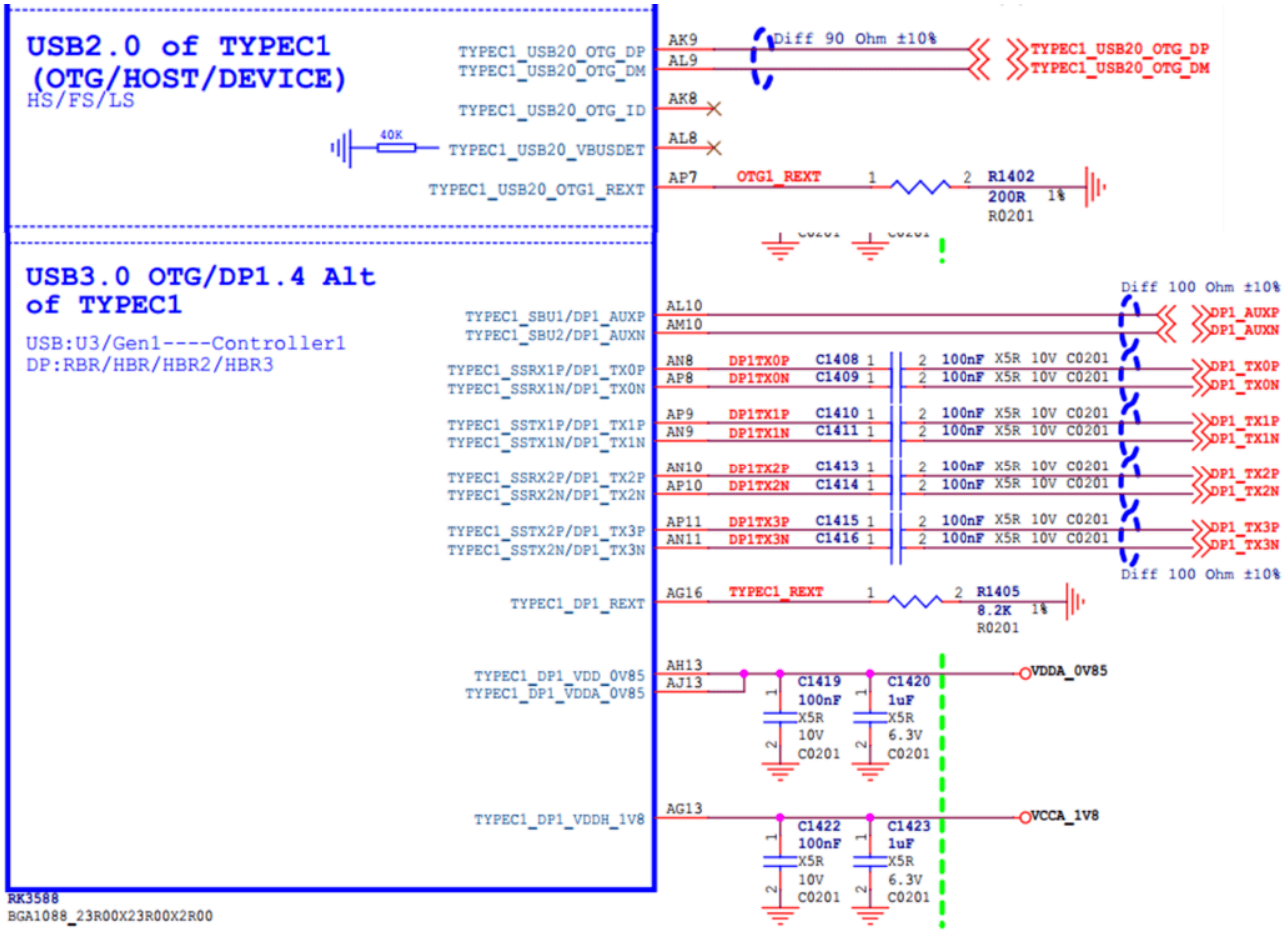

Figure 22 RK3588 Type-A USB 2.0/DP Circuit

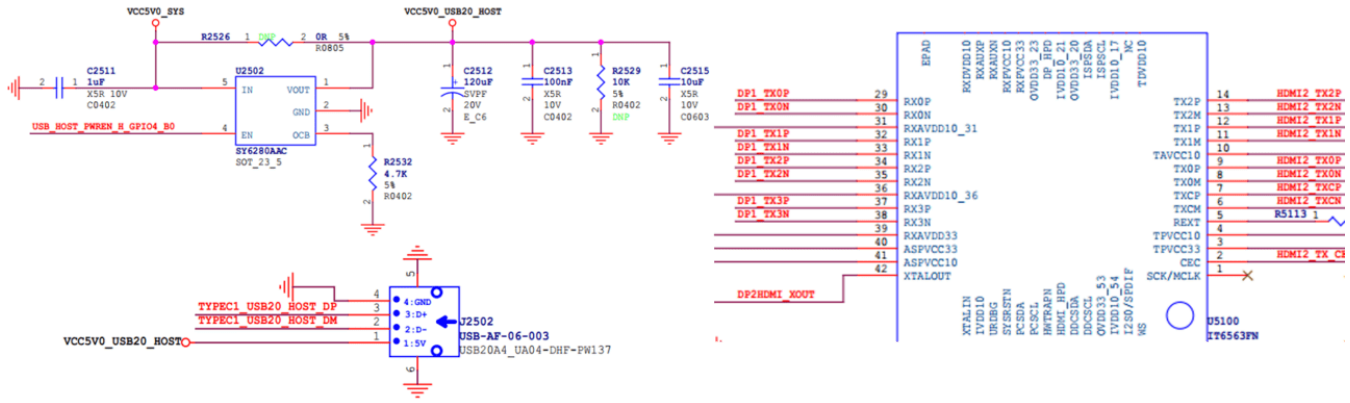

Figure 23 RK3588 Type-A USB2/DP to HDMI Interface

4.3.6 Type-A USB 3.1 Hardware Circuit

This solution is applicable to RK3588 Type-C0, Type-C1, and USB 3.1 HOST2. For the hardware circuit design corresponding to Type-C0/C1's Type-A USB 3.1, please refer to the USB 3.1 circuit design section of Type-A USB 3.1/DP Hardware Circuit.

This section focuses on the hardware circuit design of Type-A USB 3.1 corresponding to USB 3.1 HOST2.

In the previous sections of this development guide Introduction to RK3588 USB Controller and PHY, it has been mentioned that the USB 3.1 HOST_2 (also known as USB30_2) controller does not come with a USB 2.0 PHY, so USB 3.1 HOST_2 needs to be combined with USB 2.0 HOST0/1 to form a complete USB 3.1 interface function.

Taking the RK3588 EVB2 USB30_2 HOST hardware circuit design as an example.

1. USB 3.1 HOST2 forms a complete USB 3.1 interface with USB 2.0 HOST1;
2. According to the chip design of RK3588, USB 3.1 HOST2 can also be combined with the USB 2.0 of Type-C0/1, but considering that Type-C0/C1 may also be used as a USB Device, and the Device and Host functions of the USB 3.1 OTG controller cannot be used simultaneously[2], therefore, to simplify the design of software and hardware, it is recommended that USB 3.1 HOST2 only be combined with USB 2.0 HOST0/1;
3. The DTS configuration corresponding to Type-A USB 3.1 please refer to Type-A USB 3.1 DTS Configuration.

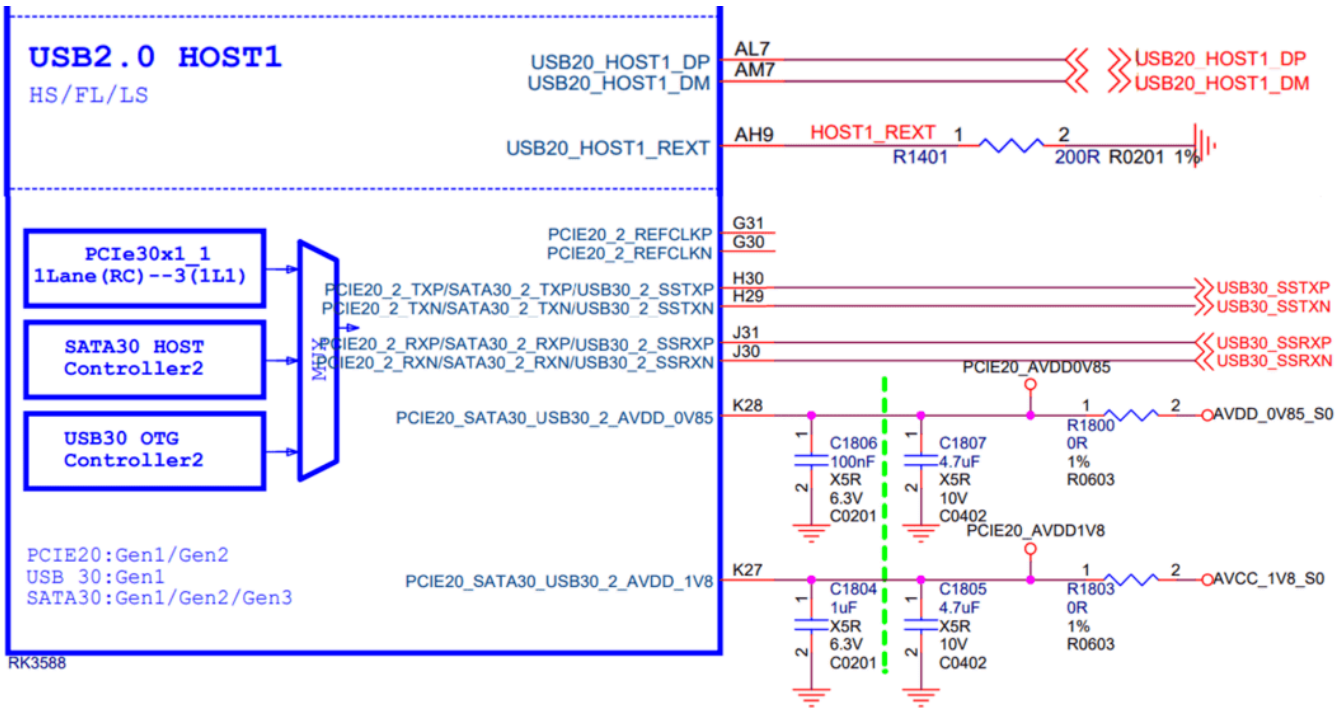

Figure 24 RK3588 USB 3.1 HOST2 Circuit

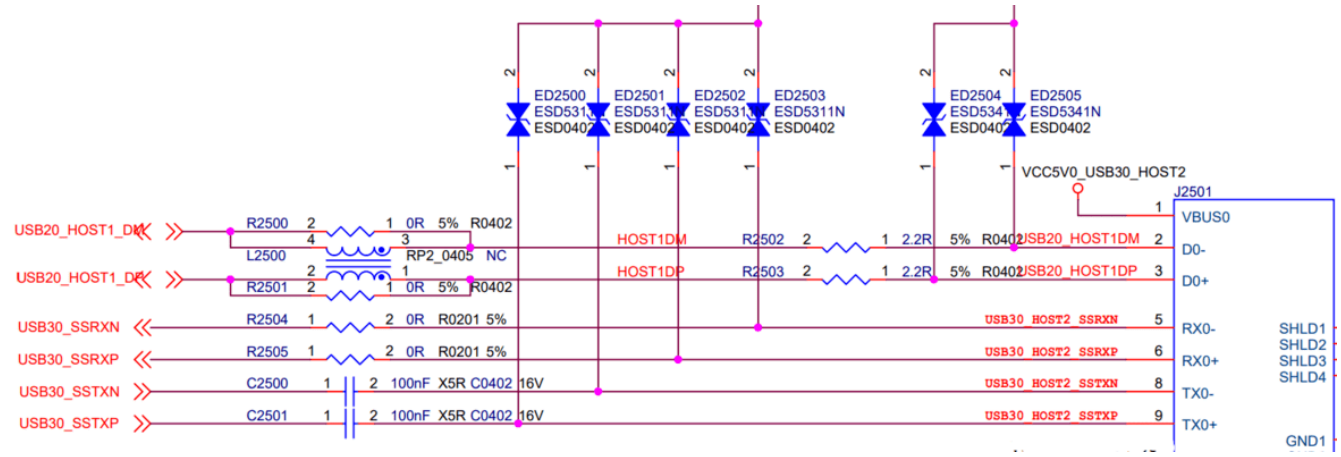

Figure 25 RK3588 USB 3.1 HOST2 Interface

4.3.7 Type-A USB 2.0 Hardware Circuit

This solution is applicable to RK3588 USB 2.0 HOST0/1.

Taking the hardware circuit design of RK3588 EVB1 USB 2.0 HOST0/1 as an example.

1. USB 2.0 HOST0/1 share the VBUS power supply (VCC5V0_USB20_HOST), controlled by GPIO to regulate the output of the voltage regulator chip SY6280AAC for VBUS. The maximum output current is determined by the resistor connected to the OCB pin of the voltage regulator chip, with the maximum current Ilim(A)=6800/Rset(ohm), as shown in Figure 26 below, the VBUS output current limit is 1.45A;
2. For the DTS configuration corresponding to Type-A USB 2.0, please refer to Type-A USB 2.0 DTS Configuration;

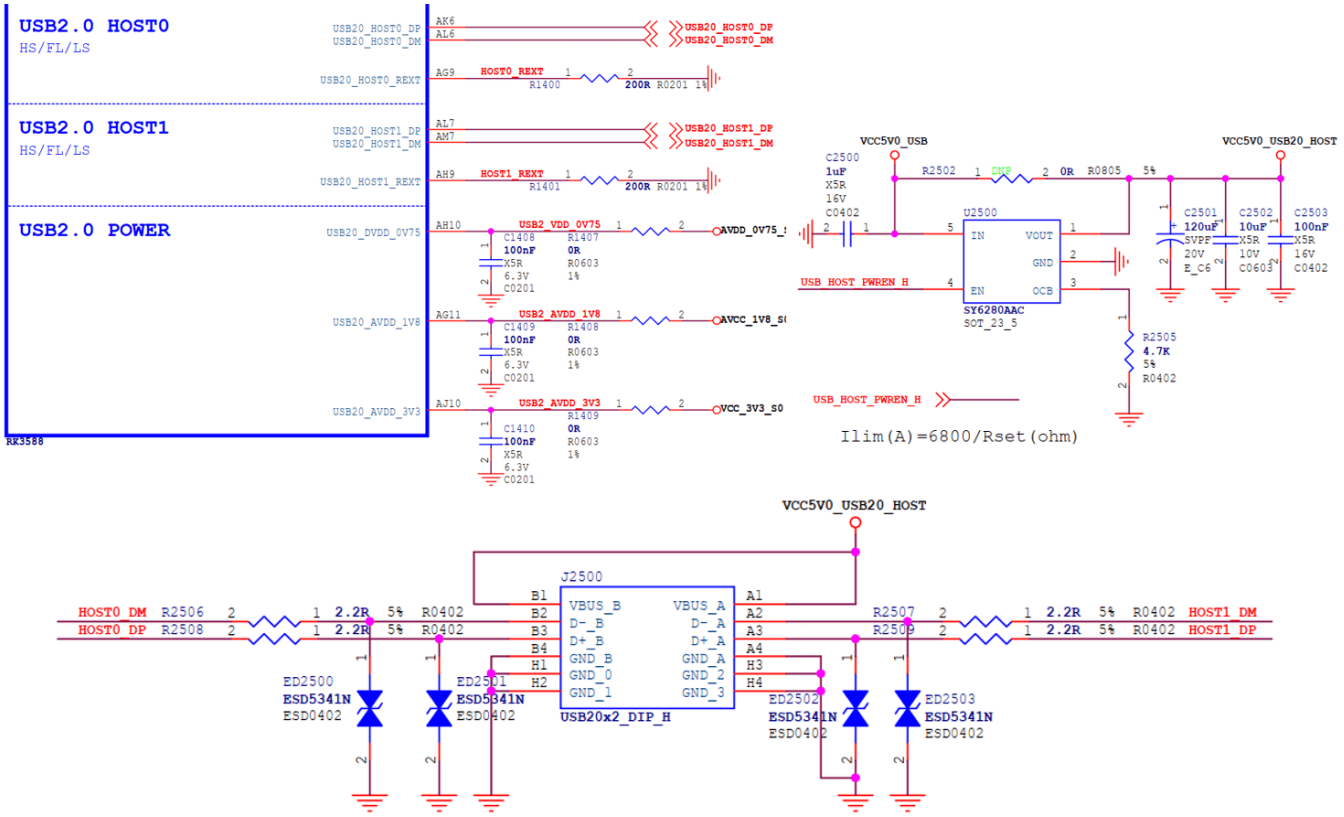

Figure 26 RK3588 Type-A USB 2.0 HOST Circuit

5. RK3588 USB DTS Configuration

RK3588 USB DTS Configuration includes: chip-level USB controller/PHY DTSI configuration and board-level DTS configuration.

For detailed configuration methods, please refer to the kernel documentation:

1. kernel/Documentation/devicetree/bindings/usb/snps,dwc3.yaml
2. kernel/Documentation/devicetree/bindings/usb/generic-ohci.yaml
3. kernel/Documentation/devicetree/bindings/usb/generic-ehci.yaml
4. kernel/Documentation/devicetree/bindings/connector/usb-connector.yaml
5. kernel/Documentation/devicetree/bindings/phy/phy-rockchip-inno-usb2.yaml
6. kernel/Documentation/devicetree/bindings/phy/phy-rockchip-usbdp.yaml
7. kernel/Documentation/devicetree/bindings/phy/phy/phy-rockchip-naneng-combphy.txt

5.1 USB Chip-Level DTSI Configuration

The main nodes related to the USB controller and PHY in the RK3588 DTSI file are shown below. Since the USB DTSI node configuration pertains to the common resources and attributes of the USB controller and PHY, it is recommended that developers do not make changes.

The DTSI configurations for USB 3.1 OTG0, USB 2.0 HOST0/1, and USB 3.1 HOST2 are placed in `rk3588s-evb.dtsi`.

The DTSI configuration for USB 3.1 OTG1 is placed in `rk3588-evb.dts`.

The corresponding complete DTSI paths are as follows:

`arch/arm64/boot/dts/rockchip/rk3588s.dtsi`

`arch/arm64/boot/dts/rockchip/rk3588.dtsi`

Note:

All USB controllers and PHYs of RK3588/RK3588S are configured as status = "okay" in `rk3588s-evb.dtsi` and `rk3588-evb.dtsi`. If the product's board-level DTS file includes these two EVB DTSI files, it is only necessary to configure the unused USB nodes as "disabled" in the board-level DTS file.

The correspondence between USB interfaces and USB DTS nodes is shown in Table 16 below.

Table 16 Correspondence between RK3588 USB Interfaces and USB DTS Nodes

USB Interface Name (Schematic)	USB Controller DTS Node	USB PHY DTS Node
TYPEC0	usbdrd3_0 usbdrd_dwc3_0	u2phy0 u2phy0_otg usbdp_phy0 usbdp_phy0_u3
TYPEC1	usbdrd3_1 usbdrd_dwc3_1	u2phy1 u2phy1_otg usbdp_phy1 usbdp_phy1_u3
USB20_HOST0	usb_host0_ehci usb_host0_ohci	u2phy2 u2phy2_host
USB20_HOST1	usb_host1_ehci usb_host1_ohci	u2phy3 u2phy3_host
USB30_2	usbhost3_0 usbhost_dwc3_0	combphy2_psu

USB Controller DTSI Nodes are as follows:

```
#USB3.1 OTG0 Controller
usbdrd3_0: usbdrd3_0 {
        compatible = "rockchip,rk3588-dwc3", "rockchip,rk3399-dwc3";
        ......
        usbdrd_dwc3_0: usb@fc000000 {
                compatible = "snps,dwc3";
                ......
        };
};
```

```
#USB2.0 HOST0 Controller
usb_host0_ehci: usb@fc800000 {
        compatible = "generic-ehci";
        ......
};

usb_host0_ohci: usb@fc840000 {
        compatible = "generic-ohci";
        ......
};

#USB2.0 HOST1 Controller
usb_host1_ehci: usb@fc880000 {
        compatible = "generic-ehci";
        ......
};

usb_host1_ohci: usb@fc8c0000 {
        compatible = "generic-ohci";
        ......
};

#USB3.1 HOST2 Controller
usbhost3_0: usbhost3_0 {
        compatible = "rockchip,rk3588-dwc3", "rockchip,rk3399-dwc3";
        ......
        usbhost_dwc3_0: usb@fcd00000 {
                compatible = "snps,dwc3";
                ......
        };
};

#USB3.1 OTG1 Controller
usbdrd3_1: usbdrd3_1 {
        compatible = "rockchip,rk3588-dwc3", "rockchip,rk3399-dwc3";
        ......
        usbdrd_dwc3_1: usb@fc400000 {
                compatible = "snps,dwc3";
                ......
        };
};
```

USB PHY DTSI Nodes are as follows:

Note: USB PHYs and USB controllers have a one-to-one correspondence and need to be configured in pairs. Inside the chip, the connection relationship between USB PHY and controller, please refer to RK3588 USB Controller and PHY Introduction Figure 1 and Table 16. In the DTSI node, the corresponding USB PHY is associated through the "phys" attribute of the USB controller node.

```
#USB2.0 PHY0
usb2phy0_grf: syscon@fd5d0000 {
                compatible = "rockchip,rk3588-usb2phy-grf", "syscon",
                             "simple-mfd";
```

```
                ......
                u2phy0: usb2-phy@0 {
                        compatible = "rockchip,rk3588-usb2phy";
                        ......
                        u2phy0_otg: otg-port {
                                #phy-cells = <0>;
                                status = "disabled";
                        };
                };
        };

#USB2.0 PHY1
usb2phy1_grf: syscon@fd5d4000 {
    ......
};

#USB2.0 PHY2
usb2phy2_grf: syscon@fd5d8000 {
    ......
};

#USB2.0 PHY3
usb2phy3_grf: syscon@fd5dc000 {
    ......
};

#USB3.1/DP Combo PHY0
usbdp_phy0: phy@fed80000 {
        compatible = "rockchip,rk3588-usbdp-phy";
        ......
        usbdp_phy0_dp: dp-port {
                #phy-cells = <0>;
                status = "disabled";
        };

        usbdp_phy0_u3: u3-port {
                #phy-cells = <0>;
                status = "disabled";
        };
};

#USB3.1/DP Combo PHY1
usbdp_phy1: phy@fed90000 {
    ......
};

#USB3.1/SATA/PCIe PHY2
combphy2_psu: phy@fee20000 {
        compatible = "rockchip,rk3588-naneng-combphy";
        ......
};
```

5.2 Type-C USB 3.1/DP Full-Function DTS Configuration

Refer to the DTS configuration of Type-C0 interface in `arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi`.

```
#USB2.0 PHY configuration attribute "rockchip,typec-vbus-det", indicating support for hardware design where Type-C VBUS_DET is always pulled high
&u2phy0_otg {
        rockchip,typec-vbus-det;
};

#USB3.1/DP PHY0, configure properties "sbu1-dc-gpios" and "sbu2-dc-gpios" according to hardware design
&usbdp_phy0 {
        orientation-switch;
        svid = <0xff01>;
        sbu1-dc-gpios = <&gpio4 RK_PA6 GPIO_ACTIVE_HIGH>;
        sbu2-dc-gpios = <&gpio4 RK_PA7 GPIO_ACTIVE_HIGH>;

        port {
                #address-cells = <1>;
                #size-cells = <0>;
                usbdp_phy0_orientation_switch: endpoint@0 {
                        reg = <0>;
                        remote-endpoint = <&usbc0_orien_sw>;
                };

                usbdp_phy0_dp_altmode_mux: endpoint@1 {
                        reg = <1>;
                        remote-endpoint = <&dp_altmode_mux>;
                };
        };
};

#USB3.1 OTG0 Controller
&usbdrd_dwc3_0 {
        dr_mode = "otg";
        usb-role-switch;
        port {
                #address-cells = <1>;
                #size-cells = <0>;
                dwc3_0_role_switch: endpoint@0 {
                        reg = <0>;
                        remote-endpoint = <&usbc0_role_sw>;
                };
        };
};

#VBUS GPIO configuration, controlled by the Type-C controller chip driver
vbus5v0_typec: vbus5v0-typec {
        compatible = "regulator-fixed";
        regulator-name = "vbus5v0_typec";
```

```
        regulator-min-microvolt = <5000000>;
        regulator-max-microvolt = <5000000>;
        enable-active-high;
        gpio = <&gpio4 RK_PD0 GPIO_ACTIVE_HIGH>;
        vin-supply = <&vcc5v0_usb>;
        pinctrl-names = "default";
        pinctrl-0 = <&typec5v_pwren>;
};

#Configure external Type-C controller chip FUSB302
#Configure properties "I2C/interrupts/vbus-supply/usb_con" according to actual hardware
design
&i2c2 {
        status = "okay";
        usbc0: fusb302@22 {
                compatible = "fcs,fusb302";
                reg = <0x22>;
                interrupt-parent = <&gpio3>;
                interrupts = <RK_PB4 IRQ_TYPE_LEVEL_LOW>;
                pinctrl-names = "default";
                pinctrl-0 = <&usbc0_int>;
                vbus-supply = <&vbus5v0_typec>;
                status = "okay";

                ports {
                        #address-cells = <1>;
                        #size-cells = <0>;

                        port@0 {
                                reg = <0>;
                                usbc0_role_sw: endpoint@0 {
                                        remote-endpoint = <&dwc3_0_role_switch>;
                                };
                        };
                };

                usb_con: connector {
                        compatible = "usb-c-connector";
                        label = "USB-C";
                        data-role = "dual";
                        power-role = "dual";
                        try-power-role = "sink";
                        op-sink-microwatt = <1000000>;
                        sink-pdos =
                                <PDO_FIXED(5000, 1000, PDO_FIXED_USB_COMM)>;
                        source-pdos =
                                <PDO_FIXED(5000, 3000, PDO_FIXED_USB_COMM)>;

                        altmodes {
                                #address-cells = <1>;
                                #size-cells = <0>;

                                altmode@0 {
```

```
                                        reg = <0>;
                                        svid = <0xff01>;
                                        vdo = <0xffffffff>;
                                };
                        };

                        ports {
                            ......
                        };
                };
        };
};
```

Note:

If using the HUSB311 chip to replace the FUSB302 chip, simply modify the DTS configuration based on FUSB302, refer to the modification:

```
#Configure external Type-C controller chip HUSB311
&i2c2 {
        usbc0: husb311@4e {
                compatible = "hynetek,husb311";
                reg = <0x4e>;
                ......
        };
};
```

5.3 USB 3.1/DP DTS Configuration for Type-C to Type-A

Refer to `arch/arm64/boot/dts/rockchip/rk3588-evb2-lp4.dtsi` for the DTS configuration of Type-C0 to Type-A USB 3.1/DP.

```
#Configure the "phy-supply" property for USB2.0 PHY0 to control the 5V VBUS output
#Note: Using phy-supply does not allow for dynamic switching of VBUS. If the OTG exclusively
uses a GPIO and does not share it with other HOSTs, and if OTG needs to support both
Device/HOST, it should be configured as "vbus-supply = <&vcc5v0_otg>" to achieve dynamic
VBUS switching.
&u2phy0_otg {
        phy-supply = <&vcc5v0_host>;
};

#Configure the VBUS GPIO, which is controlled by the USB2.0 PHY driver
vcc5v0_host: vcc5v0-host {
        compatible = "regulator-fixed";
        regulator-name = "vcc5v0_host";
        regulator-boot-on;
        regulator-always-on;
        regulator-min-microvolt = <5000000>;
        regulator-max-microvolt = <5000000>;
        enable-active-high;
```

```
        gpio = <&gpio4 RK_PA1 GPIO_ACTIVE_HIGH>;
        vin-supply = <&vcc5v0_usb>;
        pinctrl-names = "default";
        pinctrl-0 = <&vcc5v0_host_en>;
};

#USB3.1/DP PHY0, only need to configure DP to use lane2/3, the driver will automatically
allocate lane0/1 for USB3.1 Rx/Tx
#If the hardware design uses DP on lane0/1, this should be configured as "rockchip,dp-lane-
mux = <0 1>"
#Note: In actual circuitry, even if DP is not supported, "rockchip,dp-lane-mux" must be
configured, otherwise the USBDP PHY driver cannot automatically allocate lanes to USB3.1
&usbdp_phy0 {
        rockchip,dp-lane-mux = <2 3>;
};

#USB3.1 OTG0 Controller
#Set "dr_mode" to "otg" and configure the "extcon" property to support software switching
between Device/Host modes
&usbdrd_dwc3_0 {
        dr_mode = "otg";
        extcon = <&u2phy0>;
        status = "okay";
};
```

5.4 USB 2.0/DP DTS Configuration for Type-C to Type-A

Refer to `arch/arm64/boot/dts/rockchip/rk3588-nvr-demo.dtsi` for the DTS configuration of Type-C1 to Type-A USB 2.0/DP.

```
#Configure the "phy-supply" property for USB2.0 PHY1 to control the 5V VBUS output
&u2phy1_otg {
        phy-supply = <&vcc5v0_host>;
        status = "okay";
};

#Configure the VBUS GPIO, controlled by the USB2.0 PHY driver
vcc5v0_host: vcc5v0-host-regulator {
        compatible = "regulator-fixed";
        regulator-name = "vcc5v0_host";
        regulator-boot-on;
        regulator-always-on;
        regulator-min-microvolt = <5000000>;
        regulator-max-microvolt = <5000000>;
        enable-active-high;
        gpio = <&gpio4 RK_PB0 GPIO_ACTIVE_HIGH>;
        vin-supply = <&vcc5v0_sys>;
        pinctrl-names = "default";
        pinctrl-0 = <&vcc5v0_host_en>;
};
```

```
#USB3.1/DP PHY1, configure DP to use lanes 0/1/2/3
#Adjust the "rockchip,dp-lane-mux" property according to the actual hardware design
#Set the "maximum-speed" property to inform the USBDP driver to limit USB to USB2.0 only
&usbdp_phy1 {
        maximum-speed = "high-speed";
        rockchip,dp-lane-mux = < 0 1 2 3 >;
        status = "okay";
};

&usbdp_phy1_dp {
        status = "okay";
};

&usbdp_phy1_u3 {
        status = "okay";
};

#Set the "maximum-speed" property to inform the DWC3 driver to limit USB to USB2.0 only
&usbdrd_dwc3_1 {
        dr_mode = "host";
        maximum-speed = "high-speed";
        status = "okay";
        snps,dis_u2_susphy_quirk;
        snps,usb2-lpm-disable;
};
```

5.5 Type-C USB 2.0 only DTS Configuration

Configuration 1. Hardware circuit with an external Type-C controller chip, supporting PD

Refer to `arch/arm64/boot/dts/rockchip/rk3588s-tablet-rk806-single.dtsi` for Type-C0 USB 2.0 OTG DTS configuration

```
#USB2.0 PHY0 registers typec orientation switch, used to interact with the TCPM subsystem,
to obtain USB hot-plug information
&u2phy0 {
        orientation-switch;
        status = "okay";

        port {
                #address-cells = <1>;
                #size-cells = <0>;
                u2phy0_orientation_switch: endpoint@0 {
                        reg = <0>;
                        remote-endpoint = <&usbc0_orien_sw>;
                };
        };
};

#USB2.0 PHY0 OTG configuration
```

```
#Configure attribute "rockchip,sel-pipe-phystatus", indicating the selection of GRF control pipe phystatus, replacing the control of USBDP PHY
#Configure attribute "rockchip,typec-vbus-det", indicating support for Type-C VBUS_DET hardware design with a constant pull-up
#Configure attribute "rockchip,dis-u2-susphy", indicating the disabling of USB2 PHY driver's dynamic entry into suspend mode
&u2phy0_otg {
        rockchip,sel-pipe-phystatus;
        rockchip,typec-vbus-det;
        rockchip,dis-u2-susphy;
        status = "okay";
};

#Disable all related nodes of USBDP PHY0, keeping USBDP PHY0 in an uninitialized state to achieve the lowest power consumption
&usbdp_phy0 {
        status = "disabled";
};

&usbdp_phy0_dp {
        status = "disabled";
};

&usbdp_phy0_u3 {
        status = "disabled";
};

&dp0 {
        status = "disabled";
};

&usbdrd3_0 {
        status = "okay";
}

#Configure USB3.1 OTG0 Controller
#Configure "phys = <&u2phy0_otg>", i.e., not referencing USBDP PHY
#Configure maximum-speed = "high-speed", notifying the DWC3 driver to restrict USB to USB2.0 only
#Configure "snps,dis_u2_susphy_quirk", disabling the controller's hardware suspend usb2 phy function, improving USB communication stability
#Configure "snps,usb2-lpm-disable", disabling the controller's Host mode LPM function, improving USB peripheral compatibility
&usbdrd_dwc3_0 {
        dr_mode = "otg";
        status = "okay";

        maximum-speed = "high-speed";
        phys = <&u2phy0_otg>;
        phy-names = "usb2-phy";
        usb-role-switch;
        snps,dis_u2_susphy_quirk;
        snps,usb2-lpm-disable;
```

```
        port {
                #address-cells = <1>;
                #size-cells = <0>;
                dwc3_0_role_switch: endpoint@0 {
                        reg = <0>;
                        remote-endpoint = <&usbc0_role_sw>;
                };
        };
};

#Configure external Type-C controller chip FUSB302
#Need to configure "I2C/interrupts/vbus-supply/usb_con" properties according to the actual
hardware design
#Need to configure the property remote-endpoint = <&u2phy0_orientation_switch> for
usbc0_orien_sw
&i2c8 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&i2c8m2_xfer>;

        usbc0: fusb302@22 {
                compatible = "fcs,fusb302";
                reg = <0x22>;
                interrupt-parent = <&gpio0>;
                interrupts = <RK_PC4 IRQ_TYPE_LEVEL_LOW>;
                pinctrl-names = "default";
                pinctrl-0 = <&usbc0_int>;
                vbus-supply = <&vbus5v0_typec>;
                status = "okay";

                ports {
                        #address-cells = <1>;
                        #size-cells = <0>;

                        port@0 {
                                reg = <0>;
                                usbc0_role_sw: endpoint@0 {
                                        remote-endpoint = <&dwc3_0_role_switch>;
                                };
                        };
                };

                usb_con: connector {
                        compatible = "usb-c-connector";
                        label = "USB-C";
                        data-role = "dual";
                        power-role = "dual";
                        ......
                        ports {
                                #address-cells = <1>;
                                #size-cells = <0>;

                                port@0 {
```

```
                                        reg = <0>;
                                        usbc0_orien_sw: endpoint {
                                                remote-endpoint =
<&u2phy0_orientation_switch>;
                                        };
                                };
                        };

                };
        };
```

Configuration 2. Hardware circuit without an external Type-C controller chip, supporting Device only

Refer to `arch/arm64/boot/dts/rockchip/rk3588-evb5-lp4.dtsi` for Type-C0 USB 2.0 Device DTS configuration

```
#USB2.0 PHY0 OTG configuration
#Configure attribute "rockchip,sel-pipe-phystatus", indicating the selection of GRF control
pipe phystatus, replacing the control of USBDP PHY
#Configure attribute "rockchip,dis-u2-susphy", indicating the disabling of USB2 PHY driver's
dynamic entry into suspend mode
&u2phy0_otg {
        rockchip,sel-pipe-phystatus;
        rockchip,dis-u2-susphy;
        status = "okay";
};

#Disable all related nodes of USBDP PHY0, keeping USBDP PHY0 in an uninitialized state to
achieve the lowest power consumption
&usbdp_phy0 {
        status = "disabled";
};

&usbdp_phy0_dp {
        status = "disabled";
};

&usbdp_phy0_u3 {
        status = "disabled";
}

#Configure USB3.1 OTG0 Controller
#Configure dr_mode = "peripheral", notifying the DWC3 driver to initialize in Device only
mode
#Configure "phys = <&u2phy0_otg>", i.e., not referencing USBDP PHY
#Configure maximum-speed = "high-speed", notifying the DWC3 driver to restrict USB to USB2.0
only
&usbdrd_dwc3_0 {
        dr_mode = "peripheral";
        phys = <&u2phy0_otg>;
        phy-names = "usb2-phy";
        maximum-speed = "high-speed";
        snps,dis_u2_susphy_quirk;
```

```
};
```

Configuration 3. Hardware circuit without an external Type-C controller chip, supporting OTG (**CC to ID level conversion circuit is required**)

```
#USB2.0 PHY0 OTG configuration
#Configure attribute "rockchip,sel-pipe-phystatus", indicating the selection of GRF control pipe phystatus, replacing the control of USBDP PHY
#Configure attribute "rockchip,dis-u2-susphy", indicating the disabling of USB2 PHY driver's dynamic entry into suspend mode
&u2phy0_otg {
        rockchip,sel-pipe-phystatus;
        rockchip,dis-u2-susphy;
        status = "okay";
};

#Disable all related nodes of USBDP PHY0, keeping USBDP PHY0 in an uninitialized state to achieve the lowest power consumption
&usbdp_phy0 {
        status = "disabled";
};

&usbdp_phy0_dp {
        status = "disabled";
};

&usbdp_phy0_u3 {
        status = "disabled";
}

#Configure USB3.1 OTG0 Controller
#Configure dr_mode = "otg"
#Configure "phys = <&u2phy0_otg>", i.e., not referencing USBDP PHY
#Configure maximum-speed = "high-speed", notifying the DWC3 driver to restrict USB to USB2.0 only
#Configure "extcon" attribute, to support automatic switching between Device/Host mode
#Configure "snps,dis_u2_susphy_quirk", disabling the controller's hardware suspend usb2 phy function, improving USB communication stability
#Configure "snps,usb2-lpm-disable", disabling the controller's Host mode LPM function, improving USB peripheral compatibility
&usbdrd_dwc3_0 {
        dr_mode = "otg";
        phys = <&u2phy0_otg>;
        phy-names = "usb2-phy";
        maximum-speed = "high-speed";
        extcon = <&u2phy0>;
        snps,dis_u2_susphy_quirk;
        snps,usb2-lpm-disable;
};
```

5.6 Type-A USB 3.1 DTS Configuration

Refer to `arch/arm64/boot/dts/rockchip/rk3588-evb2-lp4.dtsi` for the USB30_2 HOST's DTS configuration.

```
#USB2.0 PHY3 configuration for "phy-supply" attribute, used to control the VBUS output of 5V
&u2phy3_host {
        phy-supply = <&vcc5v0_host>;
}

#VBUS GPIO configuration, controlled by this GPIO in the USB2.0 PHY driver
vcc5v0_host: vcc5v0-host {
        compatible = "regulator-fixed";
        regulator-name = "vcc5v0_host";
        regulator-boot-on;
        regulator-always-on;
        regulator-min-microvolt = <5000000>;
        regulator-max-microvolt = <5000000>;
        enable-active-high;
        gpio = <&gpio4 RK_PA1 GPIO_ACTIVE_HIGH>;
        vin-supply = <&vcc5v0_usb>;
        pinctrl-names = "default";
        pinctrl-0 = <&vcc5v0_host_en>;
};

#Enable USB3.1/SATA/PCIe Combo PHY
&combphy2_psu {
        status = "okay";
};

#Configure USB3.1 HOST2 Controller
&usbhost3_0 {
        status = "okay";
};

&usbhost_dwc3_0 {
        dr_mode = "host";
        status = "okay";
};
```

5.7 USB 2.0 DTS Configuration for Type-A

Refer to the DTS configuration of USB 2.0 HOST0/1 in `arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi`.

```
#USB2.0 PHY2/3 configuration for "phy-supply" attribute, used to control 5V VBUS output
&u2phy2_host {
        phy-supply = <&vcc5v0_host>;
};
```

```
&u2phy3_host {
        phy-supply = <&vcc5v0_host>;
};

#VBUS GPIO configuration, controlled by this GPIO in the USB2.0 PHY driver
vcc5v0_host: vcc5v0-host {
        compatible = "regulator-fixed";
        regulator-name = "vcc5v0_host";
        regulator-boot-on;
        regulator-always-on;
        regulator-min-microvolt = <5000000>;
        regulator-max-microvolt = <5000000>;
        enable-active-high;
        gpio = <&gpio4 RK_PB0 GPIO_ACTIVE_HIGH>;
        vin-supply = <&vcc5v0_usb>;
        pinctrl-names = "default";
        pinctrl-0 = <&vcc5v0_host_en>;
};

#USB2.0 HOST0/1 Controller
&usb_host0_ehci {
        status = "okay";
};

&usb_host0_ohci {
        status = "okay";
};

&usb_host1_ehci {
        status = "okay";
};

&usb_host1_ohci {
        status = "okay";
};
```

5.8 Micro USB DTS Configuration

The RK3588 OTG can support the interface design of Micro USB 2.0. Take the file arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi as an example.

In the DT configuration, the following nodes and properties should be removed:

1. The usbc0 node and its child nodes;
2. The orientation-switch property and port sub-node in the usbdp_phy0 node;
3. The usb-role-switch property and port sub-node in the usbdrd_dwc3_0 node;
4. The rockchip,typec-vbus-det property in the u2phy0_otg node;
5. The usb-typec sub-node in the pinctrl node.

In the DT configuration, the following nodes and properties should be added:

1. The vcc5v0_otg node is used to control the vbus supply;
2. The u2phy0_otg node adds the vbus-supply property;
3. The usbdrd_dwc3_0 node adds the extcon = <&u2phy0>; property.

```
[...]
# VBUS GPIO Configuration, control this GPIO in the USB2.0 PHY driver
vcc5v0_otg: vcc5v0-otg {
    compatible = "regulator-fixed";
    regulator-name = "vcc5v0_otg";
    regulator-min-microvolt = <5000000>;
    regulator-max-microvolt = <5000000>;
    enable-active-high;
    gpio = <&gpio4 RK_PA7 GPIO_ACTIVE_HIGH>;
    vin-supply = <&vcc5v0_sys>;
    pinctrl-names = "default";
    pinctrl-0 = <&vcc5v0_otg_en>;
};

&pinctrl {
    usb {
        [...]
        vcc5v0_otg_en: vcc5v0-otg-en {
            rockchip,pins = <4 RK_PA7 RK_FUNC_GPIO &pcfg_pull_up>;
        };
    };
};

[...]

# Configure status according to the actual design. If neither USB3 nor DP is used, it is
recommended to close all related nodes of USBDP PHY0 to reduce power consumption
&usbdp_phy0 {
    maximum-speed = "high-speed";
    status = "okay";
};

&usbdp_phy0_dp {
    status = "okay";
};

&usbdp_phy0_u3 {
    status = "okay";
};

&u2phy0 {
    status = "okay";
};

&u2phy0_otg {
    vbus-supply = <&vcc5v0_otg>;
    rockchip,sel-pipe-phystatus;
    rockchip,dis-u2-susphy;
```

```
    status = "okay";
};

&usbdrd_dwc3_0 {
    status = "okay";
    dr_mode = "otg";
    maximum-speed = "high-speed";
    extcon = <&u2phy0>;
    snps,dis_u2_susphy_quirk;
    snps,usb2-lpm-disable;
};
```

5.9 Considerations for Linux USB DT Configuration

5.9.1 Important Attributes of USB DT Explanation

5.9.1.1 USB Controller

1. The "usb-role-switch" is exclusively used for standard Type-C interfaces (equipped with a PD controller chip), and the dr_mode attribute must be configured as "otg"; if the dr_mode is not in "otg" mode, do not configure the "usb-role-switch".
2. One of the main functions of the "extcon" attribute is to dynamically switch the OTG mode, suitable for hardware circuit designs of non-standard Type-C interfaces (equipped with a PD controller chip), such as USB Micro interfaces or Type-A interfaces, and in schemes where the controller is configured in "otg" mode, to achieve dynamic switching of the OTG mode.
3. "snps,dis_u2_susphy_quirk", disables the USB controller hardware's automatic suspend function for usb2 phy, mainly used in USB 2.0 only schemes to enhance the stability of USB communication.
4. "snps,usb2-lpm-disable", disables the LPM function of the USB controller in Host mode, mainly used in USB 2.0 only schemes to improve compatibility with USB peripherals.

5.9.1.2 USB2 PHY

1. "vbus-supply" and "phy-supply" are both used to control the VBUS output of 5V, but they have different usage scenarios. "vbus-supply" is used for the OTG port and supports dynamic switching of VBUS. "phy-supply" is used for the USB2 HOST port, and after the system powers up, VBUS 5V is always on;
2. "rockchip,sel-pipe-phystatus", this property is used to configure the GRF USB control register, selecting GRF control pipe phystatus, replacing the control of the USBDP PHY. It is mainly used for USB 2.0 only solutions. If the USBDP PHY is not enabled, this property must be added; otherwise, the USB device cannot work properly;
3. "rockchip,typec-vbus-det", used to support hardware designs where Type-C VBUS_DET is always pulled high;
4. "rockchip,dis-u2-susphy", disables the dynamic entry of the USB2 PHY driver into suspend mode, mainly used for USB 2.0 only solutions, to keep the USB2 PHY output clock for the USB controller.

5.9.1.3 USBDP Combo PHY

1. "rockchip,dp-lane-mux", in a non-full-function Type-C solution, configures the Lane number for DP mapping. DP supports 2 or 4 lanes, for example, "rockchip,dp-lane-mux = <2, 3>;" indicates that DP Lane0 is mapped to Lane2 of the USBDP PHY, and DP Lane1 is mapped to Lane3 of the USBDP PHY; similarly, "rockchip,dp-lane-mux = <0, 1, 2, 3>;" indicates that DP Lane0 is mapped to Lane0 of the USBDP PHY, and so on, following the same pattern.

6. RK3588 USB OTG Mode Switching Commands

The RK3588 SDK supports forcing the USB OTG to switch to Host mode or Peripheral mode through software methods, independent of the USB hardware circuit's OTG ID pin or Type-C interface.

The RK3588 Linux-5.10 kernel switches the USB OTG controller to operate in Peripheral mode or Host mode in the following two ways.

Note: Method 1 relies on the correct configuration of USB DTS and is only applicable to hardware circuit designs without Type-C interfaces. Method 2 has no restrictions. Therefore, when it is uncertain whether the software and hardware are correctly matched, it is recommended to use Method 2 first.

Method 1. [Legacy]

```
#1.Force host mode
  echo host > /sys/devices/platform/fd5d0000.syscon/fd5d0000.syscon:usb2-phy@0/otg_mode
#2.Force peripheral mode
  echo peripheral > /sys/devices/platform/fd5d0000.syscon/fd5d0000.syscon:usb2-
phy@0/otg_mode
```

Method 2. [New]

```
#1.Force host mode
  echo host > /sys/kernel/debug/usb/fc000000.usb/mode
#2.Force peripheral mode
  echo device > /sys/kernel/debug/usb/fc000000.usb/mode
```

7. Type-C Controller Chip Support List

Table 17 Type-C Controller Chip Support List

Type-C Controller Chip Model	Linux-4.4	Linux-4.19	Linux-5.10	Linux-6.1	Description
ANX7411	Unsupported	Unsupported	Unsupported	In Testing	Linux-6.1 software driver is supported, functionality to be further debugged.
AW35615	Supported	Supported	Supported	Supported	Recommended for use, good compatibility with DisplayPort Alt Mode Fully compatible with FUSB302
ET7301B	Supported	Supported	Supported	Supported	Fully compatible with FUSB302[note1]
ET7303	Unsupported	Supported	Supported	Supported	Hardware compatible with FUSB302, software driver highly similar to RT1711[note2]
FUSB302	Supported	Supported	Supported	Supported	The chip used by default in RK3588 EVB.
HUSB311	Unsupported	Supported	Supported	Supported	Recommended for use Hardware compatible with FUSB302, but software driver is incompatible[note3]
RT1711H	Unsupported	Supported	Supported	Supported	Hardware compatible with FUSB302, software driver highly similar to ET7303[note4]
WUSB3801	SDK not supported, used in some projects	Unsupported	Unsupported	Unsupported	Custom single-wire communication mechanism, high error rate, unable to ensure stable communication.

note1.

- Linux-4.4 does not support the TCPM software framework, so it can only support FUSB302/ET7301B, which can be directly replaced without modifying hardware or software;
- Linux-4.19/5.10 supports the TCPM software framework and TCPCI protocol, theoretically compatible with all Type-C controller chips designed based on the TCPCI standard (e.g., ET7303/HUSB311/RT1711H);

note2.

- The small package ET7303 (it is understood that the original factory does not provide a large package) has been verified on the RK platform. The kernel needs to enable CONFIG_TYPEC_ET7303 separately.
- For detailed DTS configuration, please refer to the section Type-C USB 3.1/DP Full Function DTS Configuration for the usbc0 node, only the node name, reg address, and compatible attributes need to be modified.

note3.

- FUSB302 can be directly replaced with HUSB311. The kernel needs to use CONFIG_TYPEC_HUSB311 separately, and DTS configuration notes are the same as in note2.

note4.

- RT1711H is hardware compatible with FUSB302/ET7303, and the software driver is highly similar to ET7303. The kernel needs to enable CONFIG_TYPEC_RT1711H separately, and DTS configuration notes are the same as in note2.

8. Reference Documents

1. Universal Serial Bus Type-C Cable and Connector Specification
2. Rockchip Developer Guide USB EN